

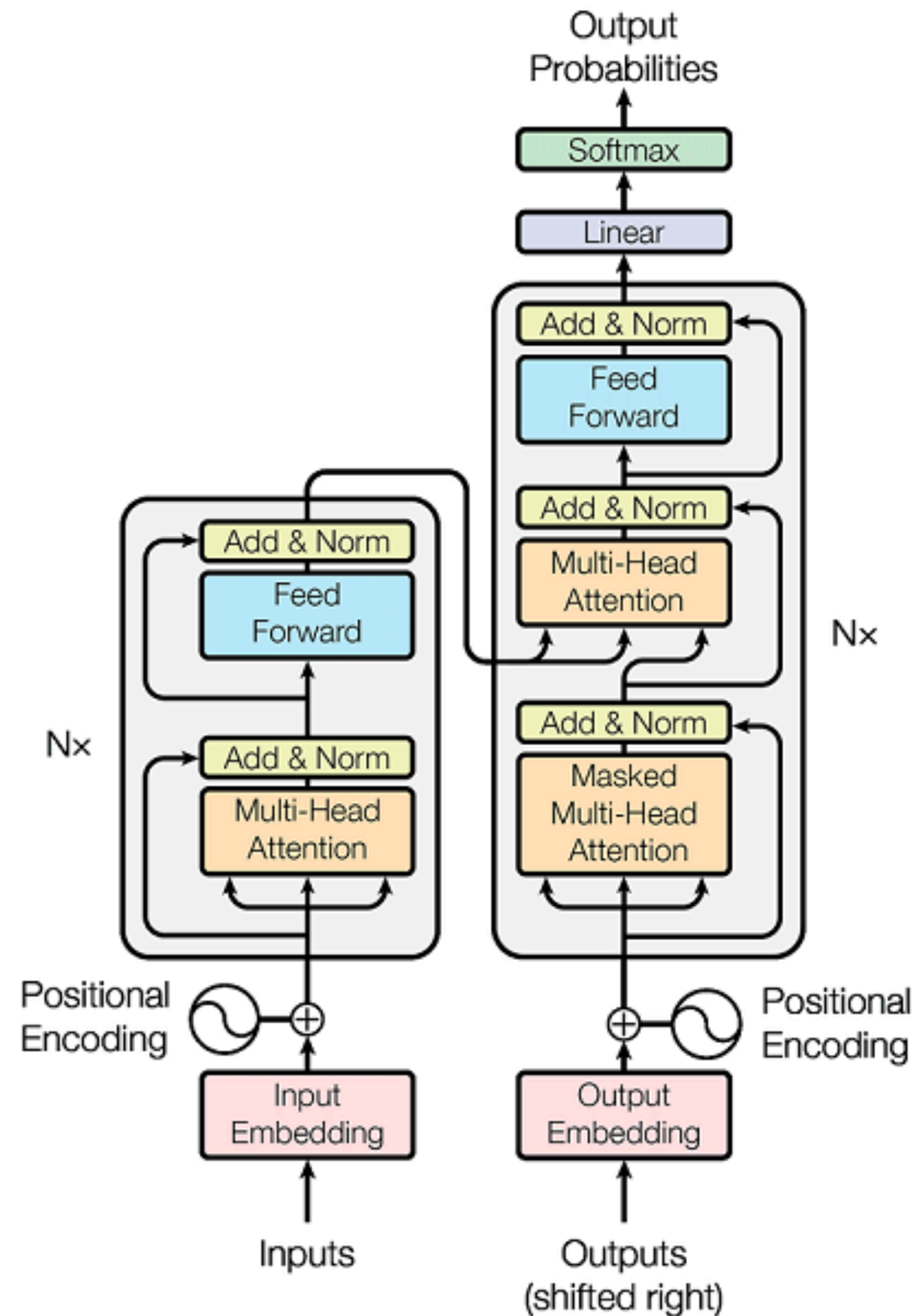


CMPT 413/713: Natural Language Processing

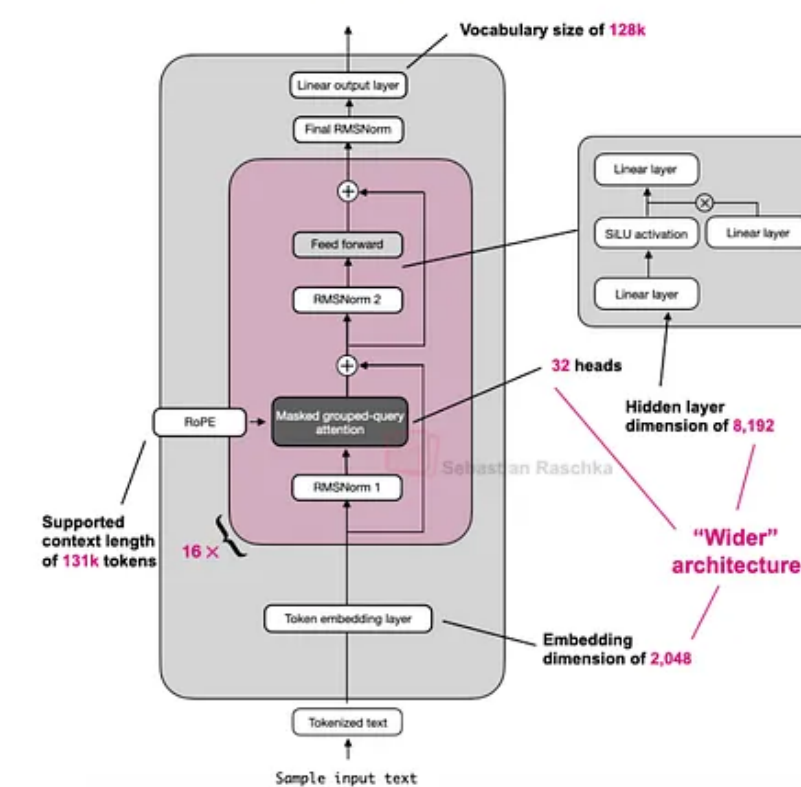
Modern LLM Architecture

Spring 2026
2026-02-26

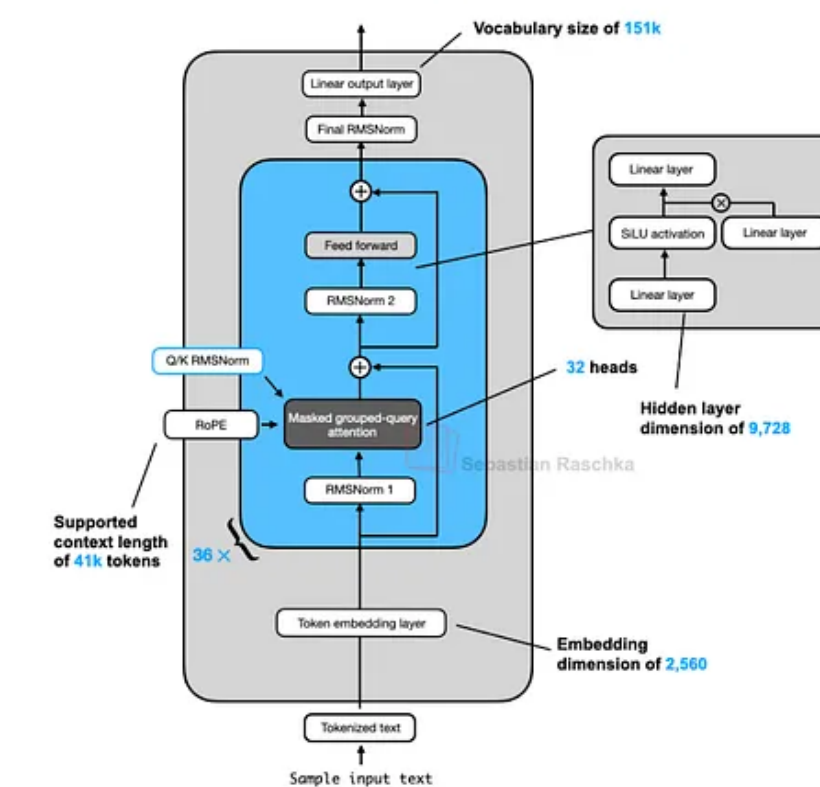
Lots of developments to the transformer block!



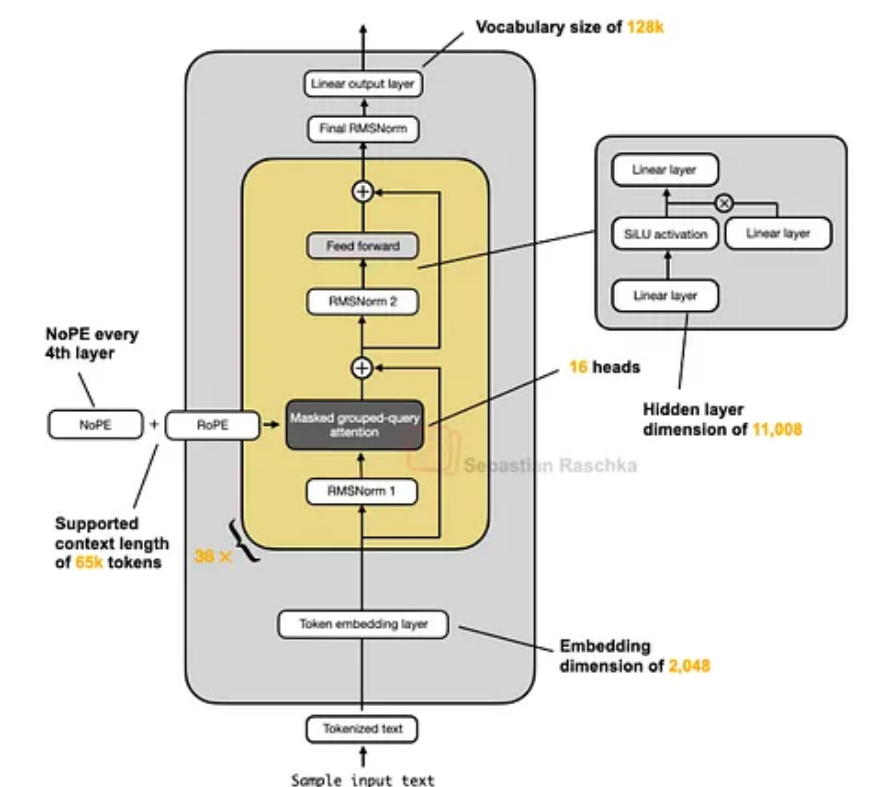
Llama 3.2 1B



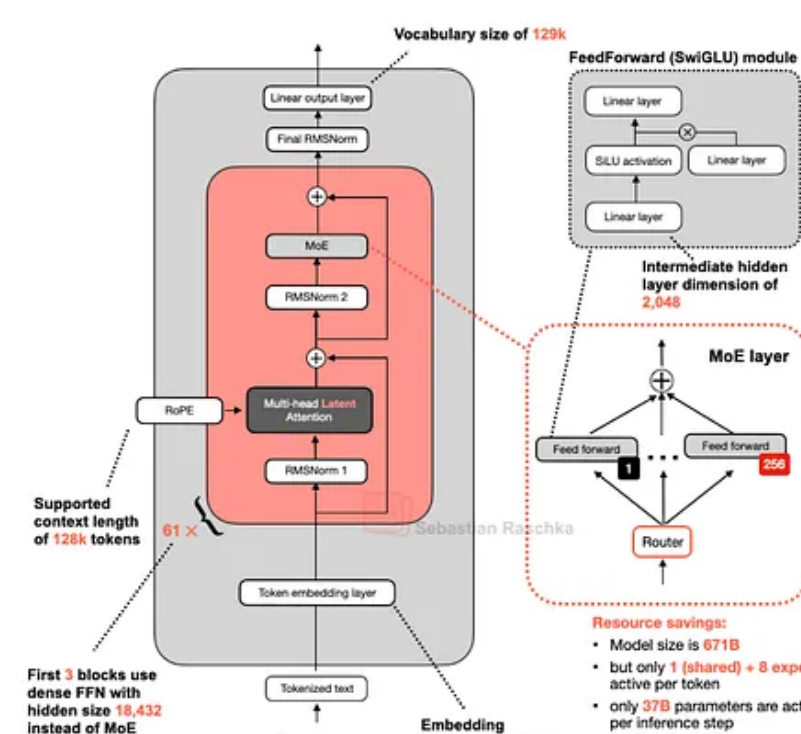
Qwen3 4B



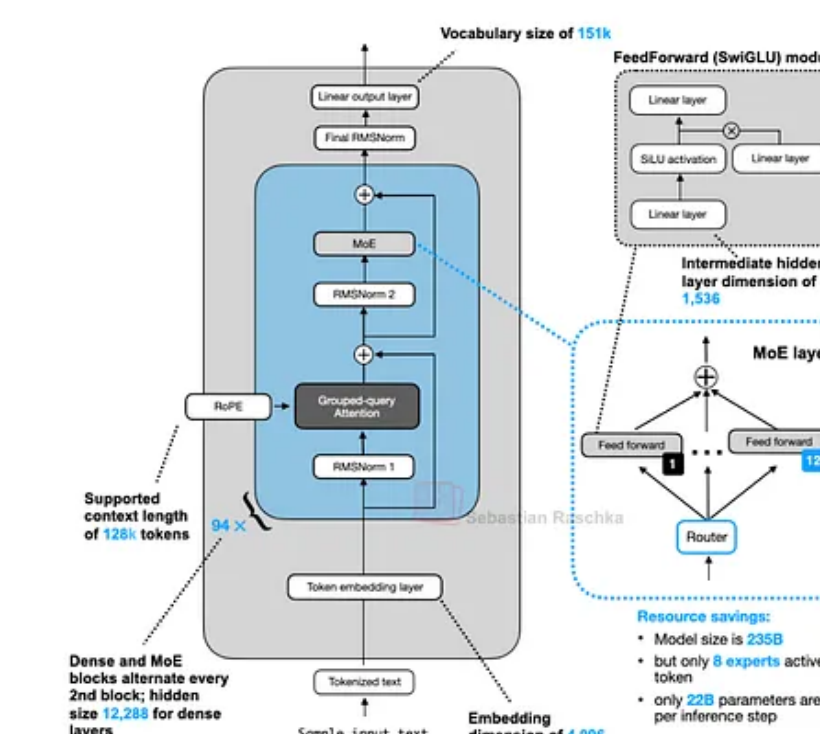
SmoLM3 3B



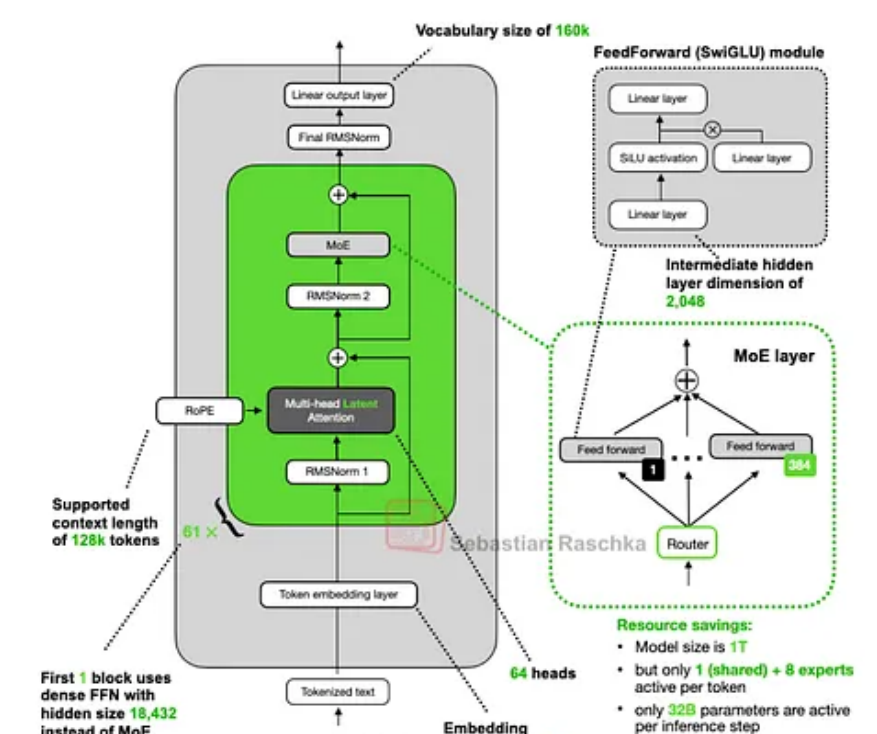
DeepSeek V3 (671B)



Qwen3 235B-A22B



Kimi K2 (1 trillion)



Transformers [2017]

<https://magazine.sebastianraschka.com/p/the-big-llm-architecture-comparison>

PaLM

PaLM: Scaling Language Modeling with Pathways, Chowdhery et al, Google, 2022

Architecture

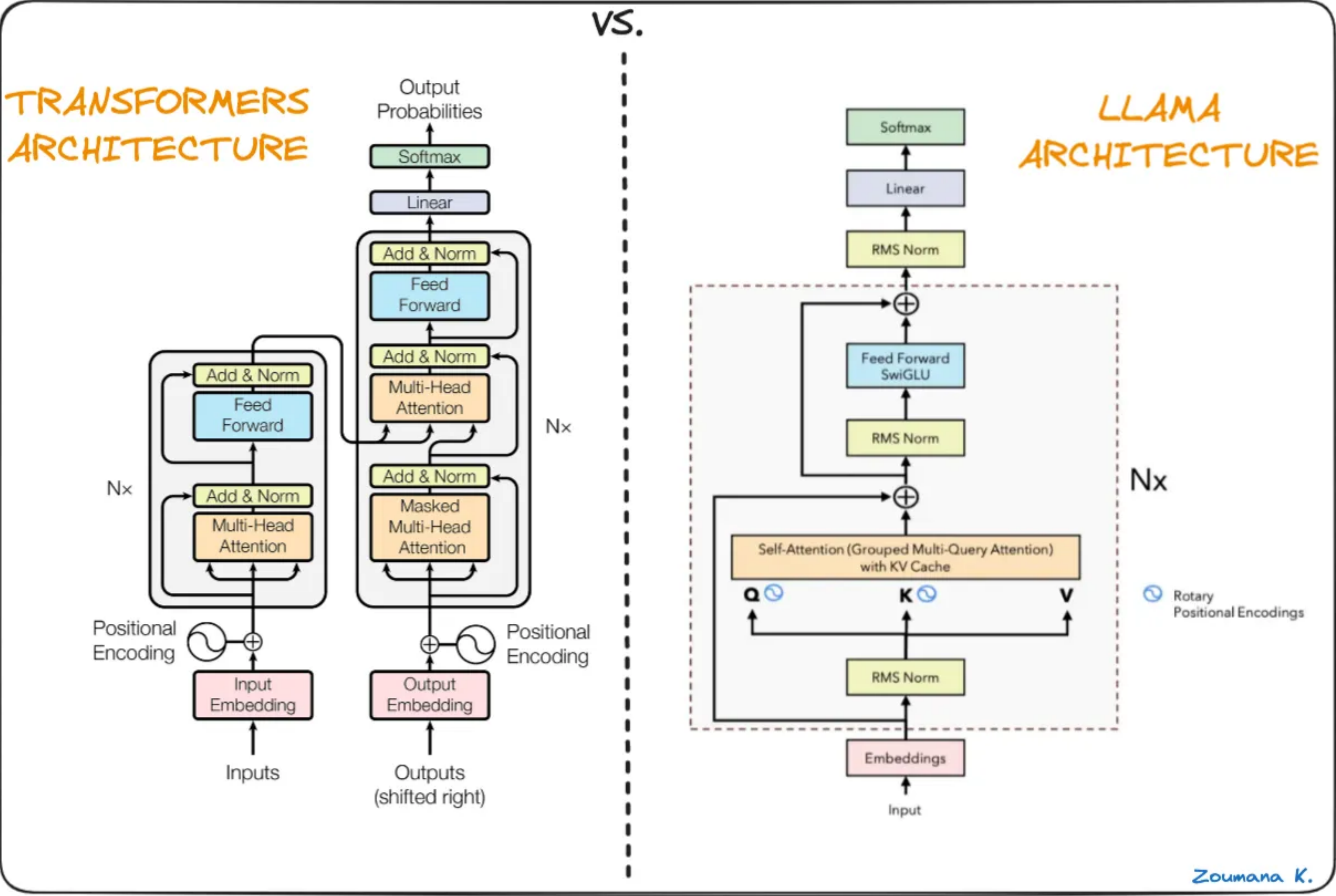
- SwiGLU activation: $\text{Swish}(xW) \otimes xV$
- Parallel layers
 - Serial: $y = x + \text{MLP}(\text{LayerNorm}(x + \text{Attention}(\text{LayerNorm}(x))))$
 - Parallel: $y = x + \text{MLP}(\text{LayerNorm}(x)) + \text{Attention}(\text{LayerNorm}(x))$
 - 15% faster training speed (degradation for small models 8B, but no degradation at 62B)
- Attention: Shared key-value across heads, query is still separately projected per head
- RoPE (rotary position) embeddings
- Shared input-output embeddings
- No biases: increased training stability
- Vocabulary: SentencePiece with 256k tokens

Training data

- 780 billion tokens of natural language + source code from github

LLaMa

	Vaswani et al.	LLaMa
Norm Position	Post	Pre
Norm Type	LayerNorm	RMSNorm
Non-linearity	ReLU	SwiGLU
Positional Encoding	Sinoidal	RoPE
Attention	Full Multi-Head Attention	Grouped Multi-Query Attention



Revisiting transformers

- Positional encoding
- Normalization
 - Layer normalization vs RMS
 - Post vs Pre-Layer norm
- Activation functions
- Attention variations

Revisiting transformers

- **Positional encoding**
- Normalization
 - Layer normalization vs RMS
 - Post vs Pre-Layer norm
- Activation functions
- Attention variations

Positional encoding

- Original transformer: fixed sinusoidal absolute embeddings
- Learned encoding
- Absolute vs relative
 - In most cases, it is the relative position between two words that matter (not their absolute position)
 - Relative encoding can be learned [Self-Attention with Relative Position Representations, Shaw et al. 2018]
- Rotary embeddings (RoPE)

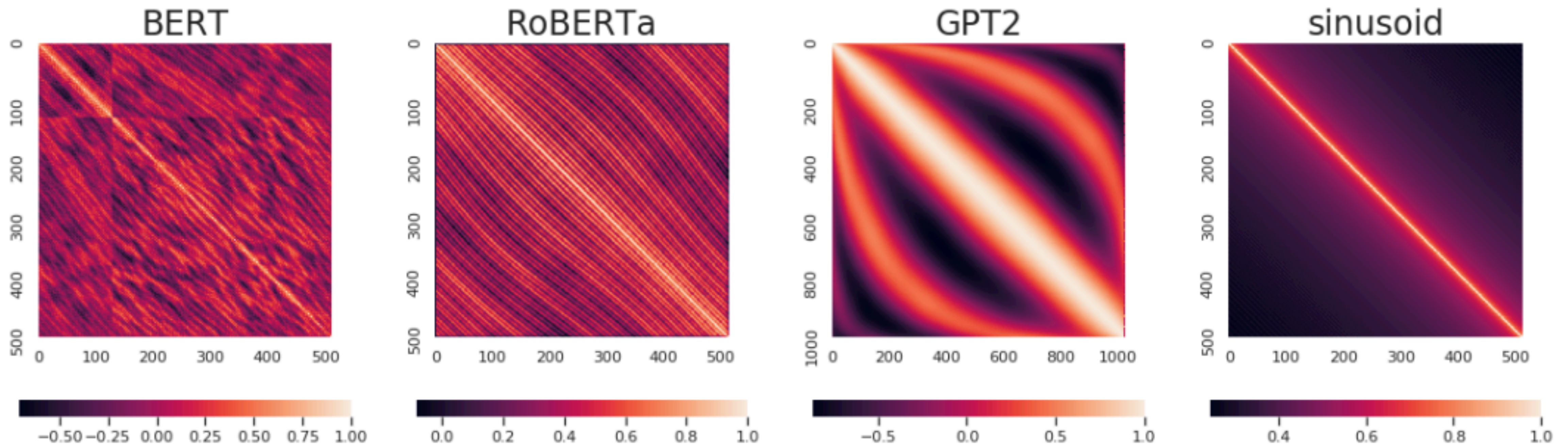
Learned encoding

- **Advantage:** Flexible, learned representations
- **Disadvantage:** bunch of extra parameters that need to be learned
- **Disadvantage:** impossible to extrapolate to longer sequences

Learned encoding

What do position embeddings learn?

- Visualize cosine similarity between position embeddings
- GPT-2 learned embeddings are quite good: can effectively predict absolute position using linear regression and relative ordering using logistic regression



Learned encoding

What do position embeddings learn?

- Visualize cosine similarity between position embeddings
- GPT-2 learned embeddings are quite good: can effectively predict absolute position using linear regression and relative ordering using logistic regression

Absolute

Type	PE	MAE
Learned	BERT	34.14
	RoBERTa	6.06
	GPT-2	1.03
Pre-Defined	sinusoid	0.0

Relative

Type	PE	Error Rate
Learned	BERT	19.72%
	RoBERTa	7.23%
	GPT-2	1.56%
Pre-Defined	sinusoid	5.08%

Relative encoding

- Learnable relative embeddings

$$f_q(\mathbf{x}_m) := \mathbf{W}_q \mathbf{x}_m$$

$$f_k(\mathbf{x}_n, n) := \mathbf{W}_k(\mathbf{x}_n + \tilde{\mathbf{p}}_r^k)$$

$$f_v(\mathbf{x}_n, n) := \mathbf{W}_v(\mathbf{x}_n + \tilde{\mathbf{p}}_r^v)$$

Self-Attention with Relative Position Representations
[Shaw et al. 2018]

- Modify attention scores to capture relative embedding

$$\mathbf{q}_m^\top \mathbf{k}_n = \mathbf{x}_m^\top \mathbf{W}_q^\top \mathbf{W}_k \mathbf{x}_n + \mathbf{x}_m^\top \mathbf{W}_q^\top \mathbf{W}_k \mathbf{p}_n + \mathbf{p}_m^\top \mathbf{W}_q^\top \mathbf{W}_k \mathbf{x}_n + \mathbf{p}_m^\top \mathbf{W}_q^\top \mathbf{W}_k \mathbf{p}_n$$

- Simplify to just learning a bias term

$$\mathbf{q}_m^\top \mathbf{k}_n = \mathbf{x}_m^\top \mathbf{W}_q^\top \mathbf{W}_k \mathbf{x}_n + b_{i,j}$$

Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer [Raffel et al. 2018]

Attention with Linear Biases (ALiBi)

- No explicit position embedding
- Bias query-key attention scores with fixed penalty that is proportional to the distance
- Allows for better extrapolation to long sequences at test time

The diagram illustrates the ALiBi attention mechanism. It shows a 5x5 matrix of query-key dot products ($q_i \cdot k_j$) being added to a 5x5 matrix of linear biases. The result is then multiplied by a scalar m .

The first matrix (Query-Key Dot Products) is:

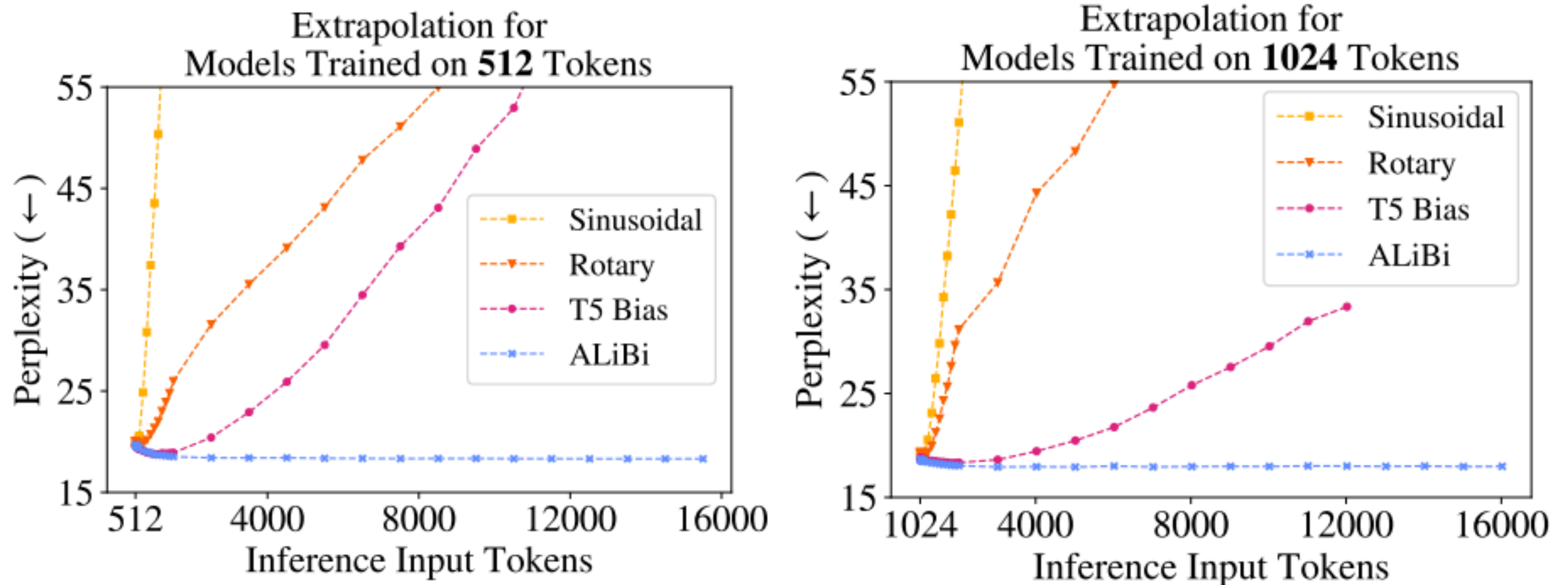
$q_1 \cdot k_1$				
$q_2 \cdot k_1$	$q_2 \cdot k_2$			
$q_3 \cdot k_1$	$q_3 \cdot k_2$	$q_3 \cdot k_3$		
$q_4 \cdot k_1$	$q_4 \cdot k_2$	$q_4 \cdot k_3$	$q_4 \cdot k_4$	
$q_5 \cdot k_1$	$q_5 \cdot k_2$	$q_5 \cdot k_3$	$q_5 \cdot k_4$	$q_5 \cdot k_5$

The second matrix (Linear Biases) is:

0				
-1	0			
-2	-1	0		
-3	-2	-1	0	
-4	-3	-2	-1	0

The operation is: $\text{Matrix 1} + \text{Matrix 2} \cdot m$

Attention with Linear Biases (ALiBi)



Rotary encoding

- Design absolute embeddings so the dot product result in function of relative position

$$f_q(\mathbf{x}_m, m) \cdot f_k(\mathbf{x}_n, n) = g(\mathbf{x}_m, \mathbf{x}_n, m - n)$$

- **Rotary Position Embedding (RoPE)**: Apply rotation to encode positional encoding (vs using addition).

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m$$

- For dimension d , break down into $d/2$ 2D subspace.
 - For vector of length d , group into $d/2$ pairs of numbers - each pair is a vector in 2D.
 - Encode positional encoding m on each pair by rotating by an angle $m\theta$

Rotary encoding

- Design absolute embeddings so the dot product result in function of relative position

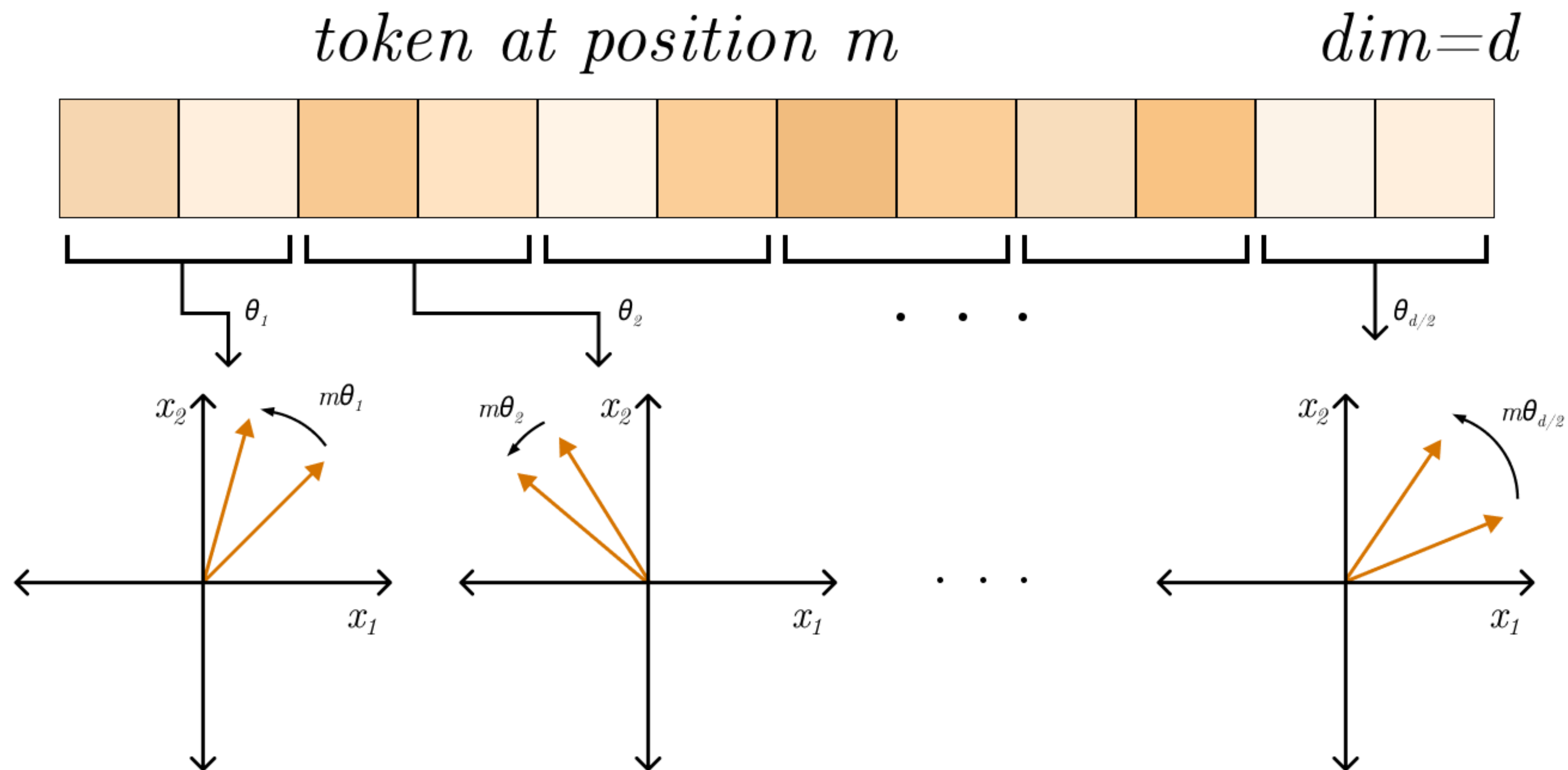
$$f_q(\mathbf{x}_m, m) \cdot f_k(\mathbf{x}_n, n) = g(\mathbf{x}_m, \mathbf{x}_n, m - n)$$

- Rotary Position Embedding (RoPE):** Apply rotation to encode positional encoding (vs using addition).

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m$$

$$R_{\Theta, m}^d = \begin{bmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{\frac{d}{2}} & -\sin m\theta_{\frac{d}{2}} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{\frac{d}{2}} & \cos m\theta_{\frac{d}{2}} \end{bmatrix}$$

Rotary encoding



Rotary encoding

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m$$

Why it works?

- For dimension d , break down into $d/2$ 2D subspace.
- For vector of length d , group into $d/2$ pairs of numbers - each pair is a vector in 2D.
- Encode positional encoding m on each pair by rotating by an angle $m\theta$

$$R_{\Theta, m}^d = \begin{bmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{\frac{d}{2}} & -\sin m\theta_{\frac{d}{2}} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{\frac{d}{2}} & \cos m\theta_{\frac{d}{2}} \end{bmatrix}$$

Rotary encoding

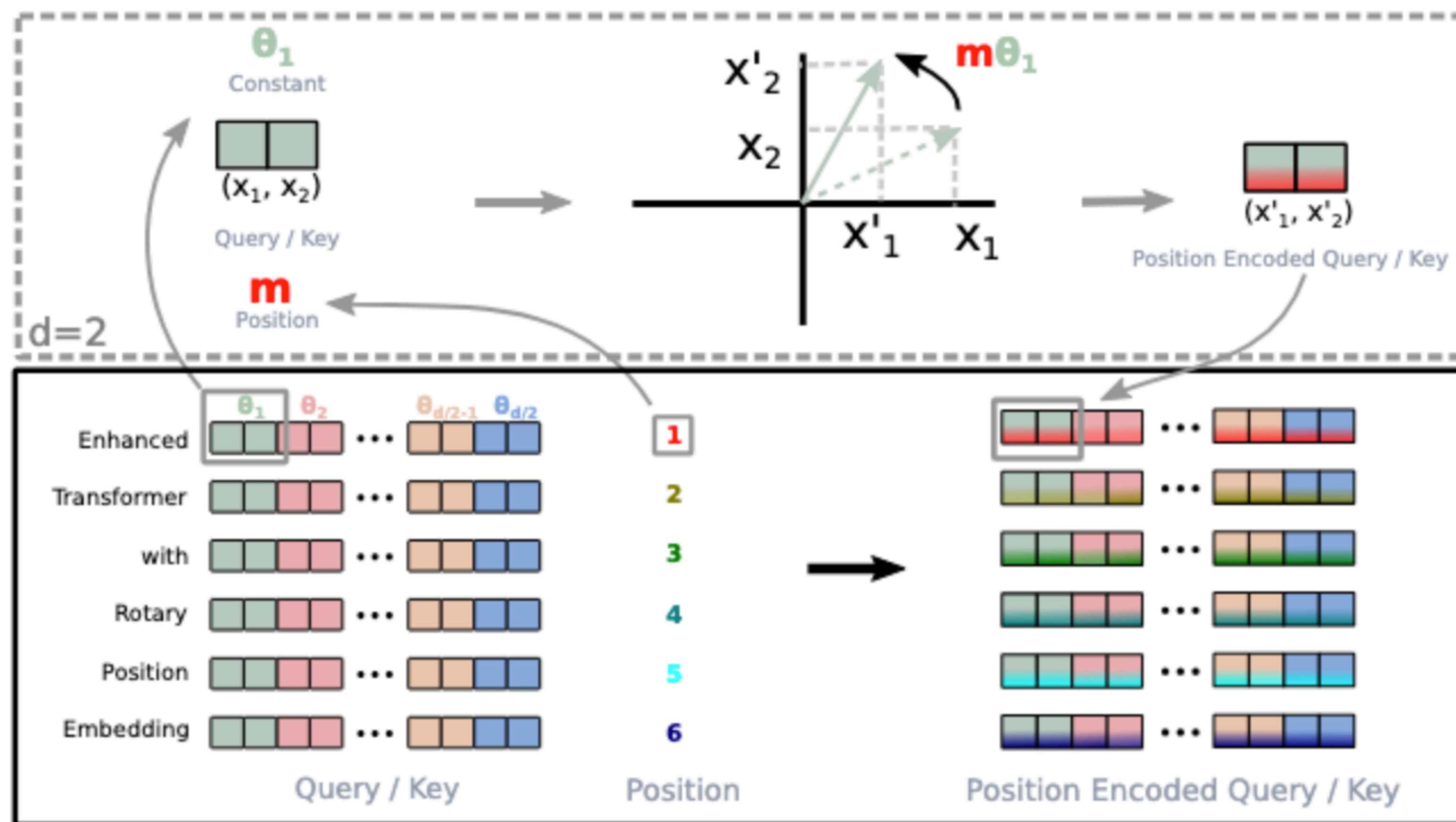
More efficient form

- With just element wise multiply and addition

$$R_{\Theta, m}^d \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_{d-1} \\ x_d \end{bmatrix} \otimes \begin{bmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \cos m\theta_2 \\ \cos m\theta_2 \\ \vdots \\ \cos m\theta_{\frac{d}{2}} \\ \cos m\theta_{\frac{d}{2}} \end{bmatrix} + \begin{bmatrix} -x_2 \\ x_1 \\ -x_4 \\ x_3 \\ \vdots \\ -x_d \\ x_{d-1} \end{bmatrix} \otimes \begin{bmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \sin m\theta_2 \\ \sin m\theta_2 \\ \vdots \\ \sin m\theta_{\frac{d}{2}} \\ \sin m\theta_{\frac{d}{2}} \end{bmatrix}$$

$$\Theta = \{\theta_i = 10000^{-2(i-1)/d}, i \in [1 \dots d/2]\}$$

Rotary encoding



RoFormer: Enhanced Transformer with Rotary Position Embedding [Su et al. 2021]

Rotary encoding

Benefits

- Absolute encoding that captures relative information
- Computationally efficient
- Integrates well into KV cache

Rotary encoding

Dot product

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m$$

$$\mathbf{q}_m^\top \mathbf{k}_n = (\mathbf{R}_{\Theta, m}^d \mathbf{W}_q \mathbf{x}_m)^\top (\mathbf{R}_{\Theta, n}^d \mathbf{W}_k \mathbf{x}_n) = \mathbf{x}_m^\top \mathbf{W}_q^\top (\mathbf{R}_{\Theta, m}^{d\top} \mathbf{R}_{\Theta, n}^d) \mathbf{W}_k \mathbf{x}_n$$

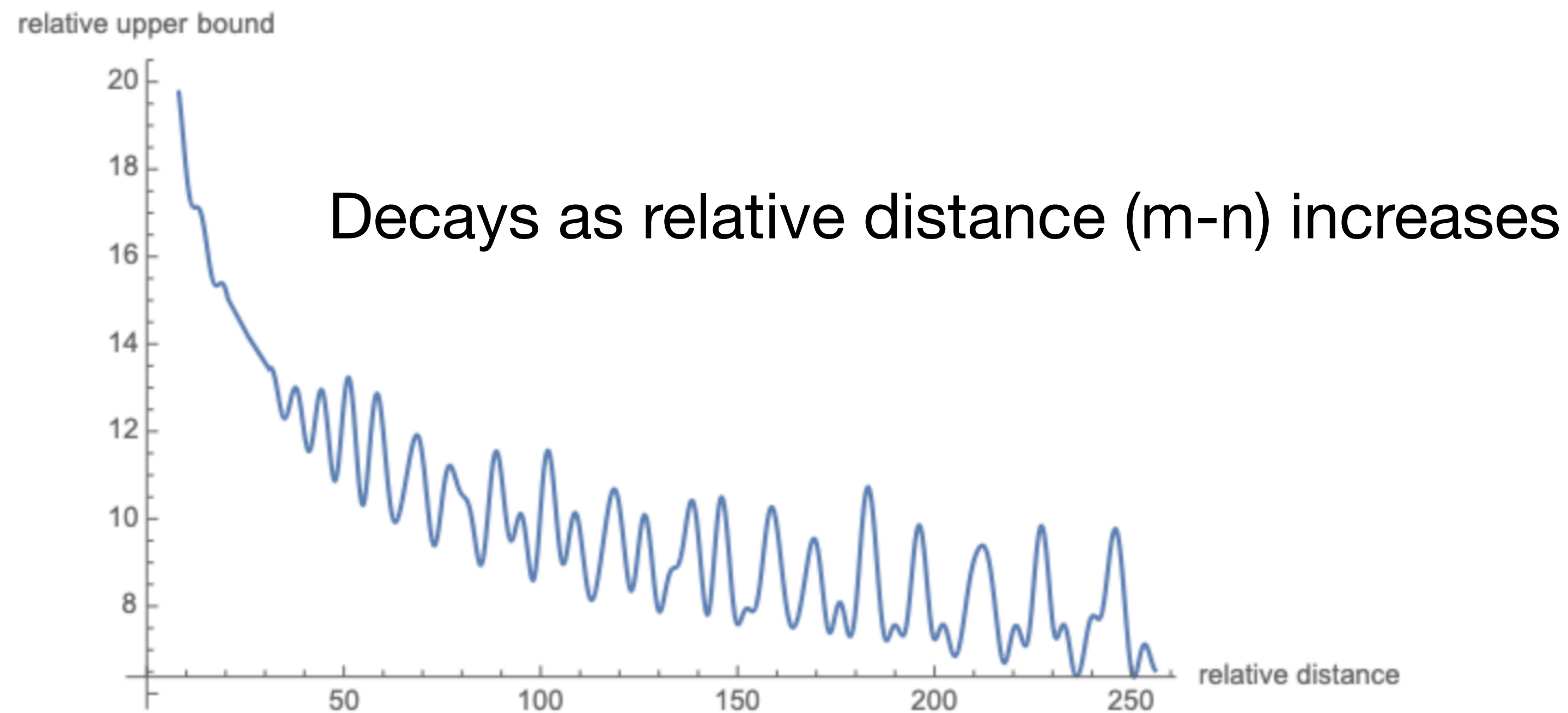


Figure 2: Long-term decay of RoPE.

Current LLMs use RoPE

- Advantage of RoPE may not hold for large models

Model size	Pos. Emb.	Performance		
		zs-main $\uparrow$	zs-small $\uparrow$	ppl-pile $\downarrow$
<i>Likely</i> threshold ($1-\sigma$)		± 1.1	± 0.4	± 0.002
1B	ALiBi	49.2	<u>43.1</u>	<u>0.895</u>
	URPE	49.6	<u>43.1</u>	0.885
	RoPE	50.0	44.2	0.883
3B	ALiBi	54.4	<u>49.8</u>	<u>0.807</u>
	RoPE	54.4	50.5	0.799

The Falcon Series of Open Language Models
[Almazrouei et al. 2023]

- But can depend on value of theta
 - Recent models use $\theta = 500K$ (vs initial $\theta = 10K$) for long context

LLaMa, OLMo

Comparing RoPE variants

Adjusted base frequency (ABF)

- RoPE with $\theta = 10K$ vs $\theta = 500K$

PE	Books	CC	Wikipedia
RoPE	6.548	6.816	3.802
RoPE PI	6.341	6.786	3.775
RoPE ABF	6.323	6.780	3.771
xPos ABF	6.331	6.780	3.771

Effective Long-Context Scaling of Foundation Models
[Xiong et al. 2023]

- Position interpolation (PI)
- xPos - Variant of RoPE

$$\mathbf{f}'(\mathbf{x}, m) = \mathbf{f}\left(\mathbf{x}, \frac{mL}{L'}\right)$$

Position interpolation:
Rescale from new length L'
to original length L

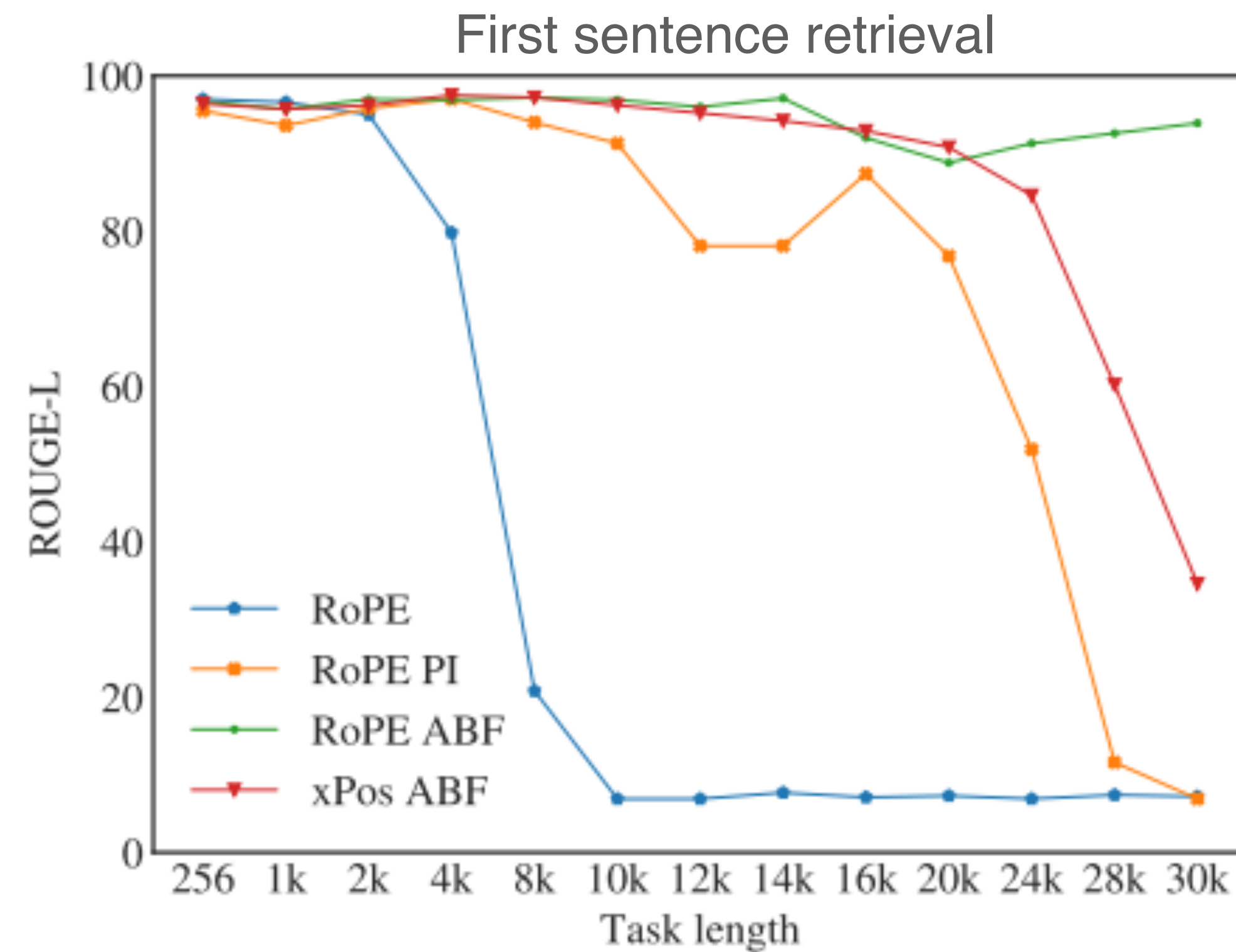
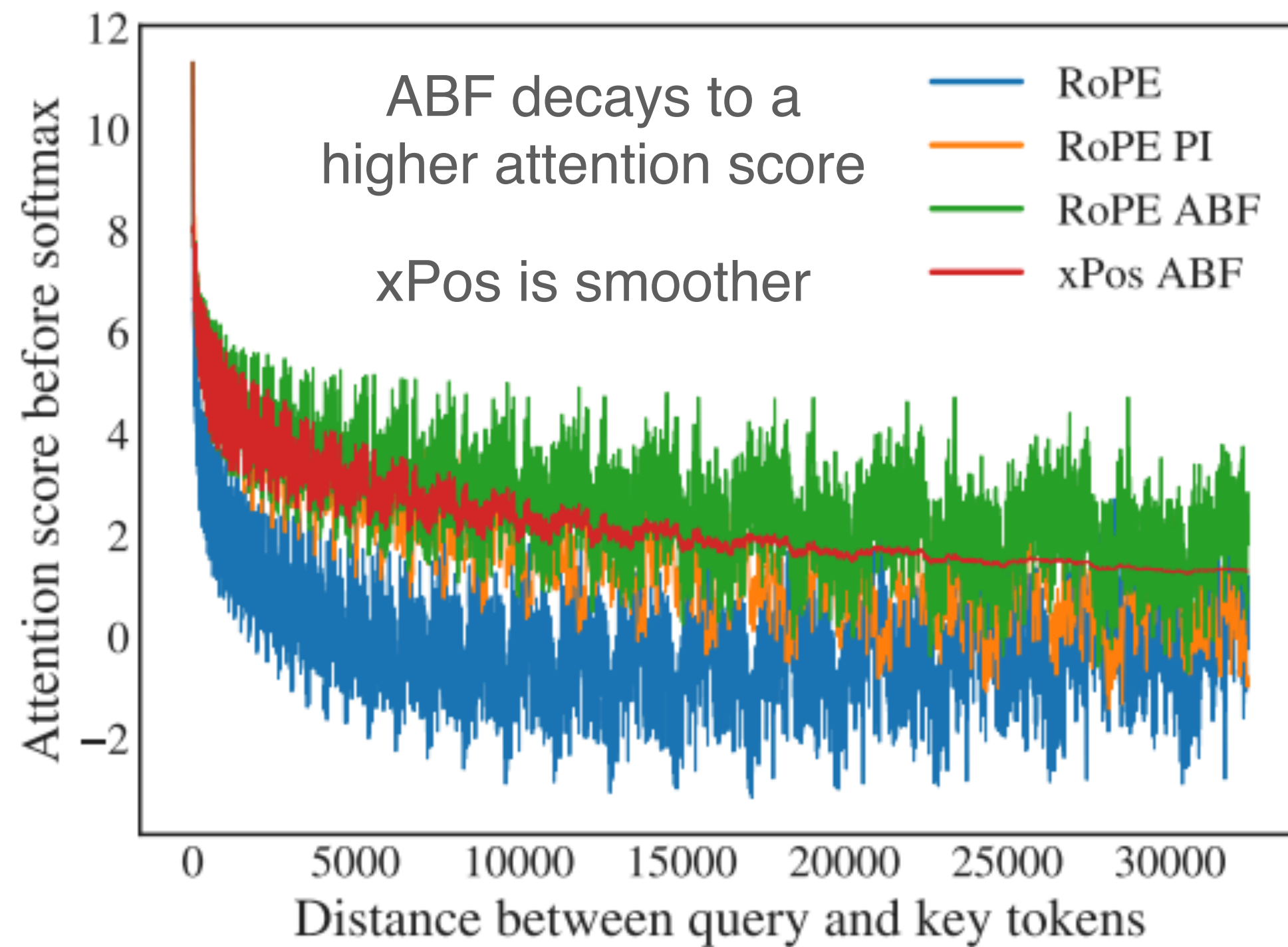
A length-extrapolatable transformer [Sun et al. 2022]

Extending Context Window of Large Language Models via Positional Interpolation [Chen et al. 2023]

Current LLMs use RoPE

Adjusted base frequency (ABF)

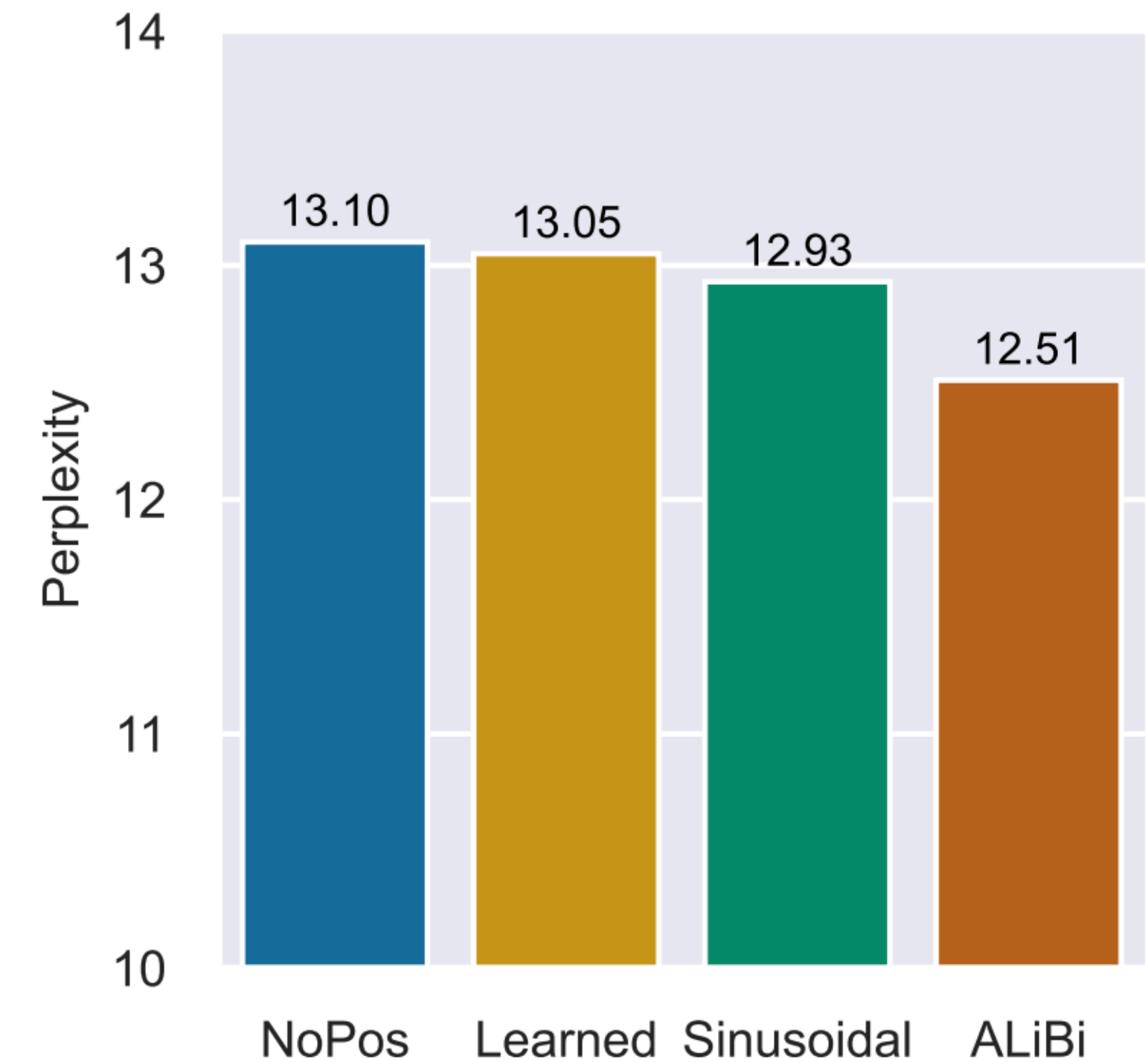
- Comparing RoPE with $\theta = 10K$ vs $\theta = 500K$



How important is positional encoding?

Can we remove them?

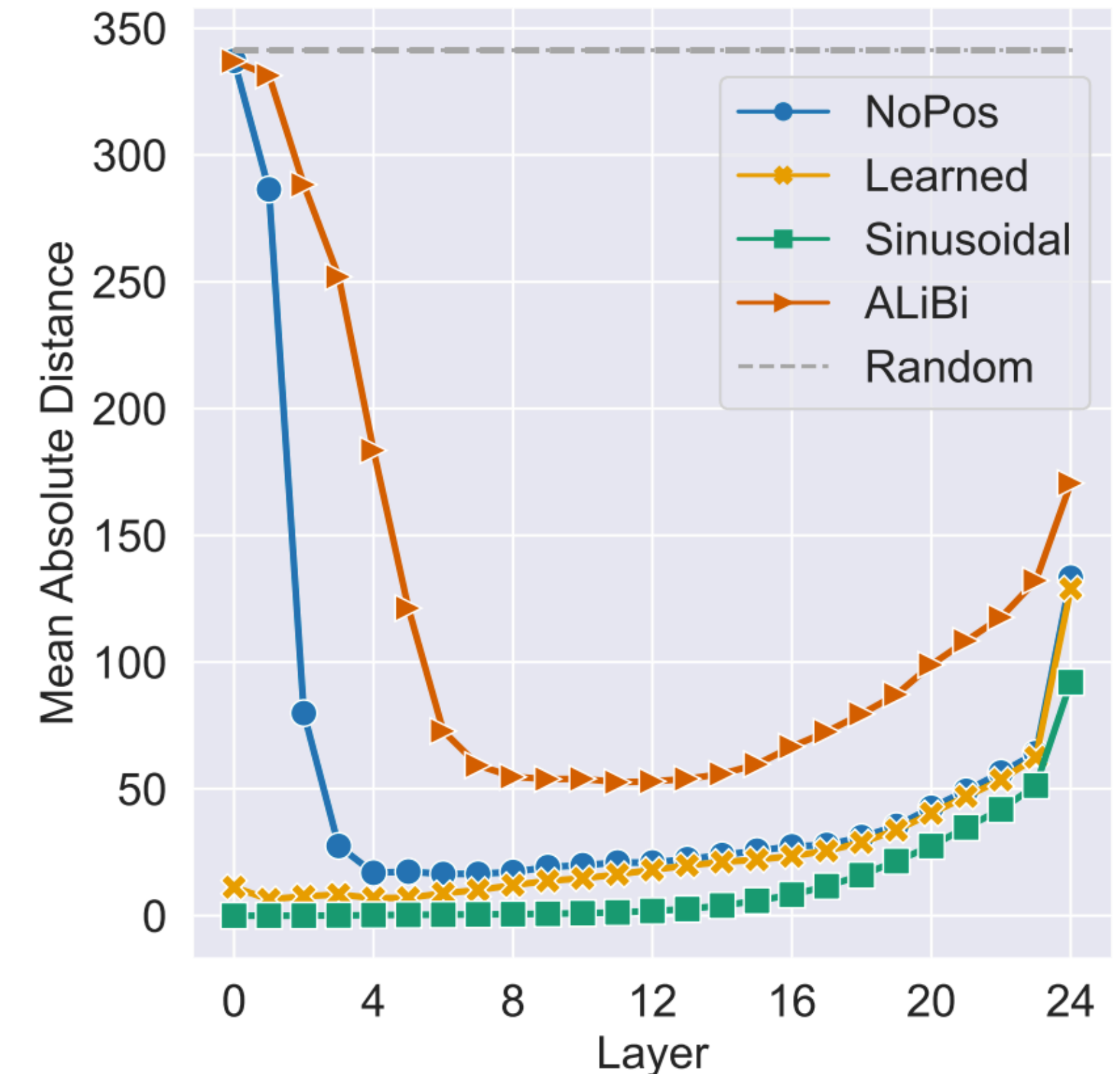
- Train 1.3B parameter autoregressive LM on the Pile
- LM perplexity comparable to learned embedding
- Hypothesis: causal attention allows for encoding of absolute position of tokens



How important is positional encoding?

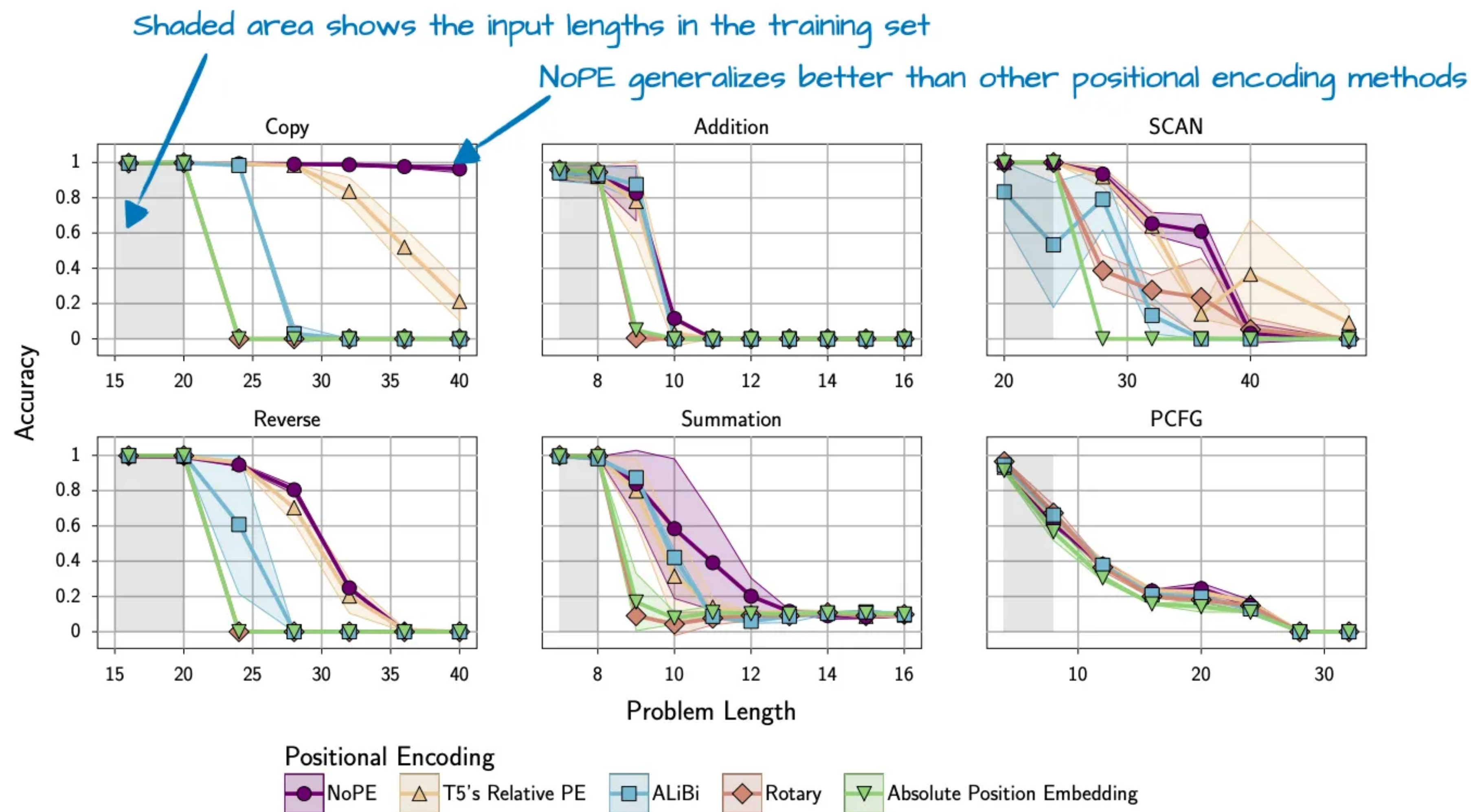
Can we remove them?

- Model can learn to encode absolute position
- Train 2-layer FF ReLU network (hooked up to transformer layer output representations) to predict absolute position (0 to 1023) of each token (as multi class classification)
- LM weights are frozen
- Later layers can predict position



NoPE (no positional embedding)

- Interleave NoPE layers with layers with positional embeddings



Revisiting transformers

- Positional encoding
- **Normalization**
 - Layer normalization vs RMS
 - Post vs Pre-Layer norm
- Activation functions
- Attention variations

Add & Norm

Residual Connections and Layer Norm

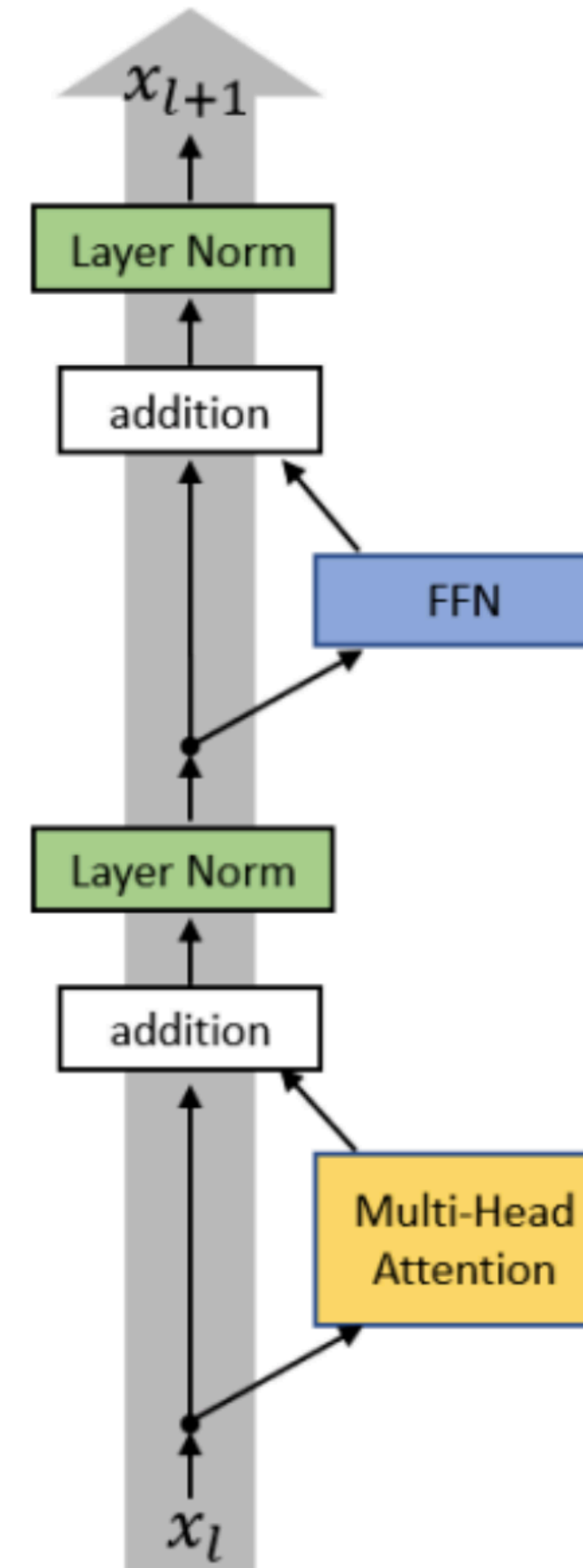
- Combine residual connection and layer norm into a single "Add & Norm" component
- Three choices:
 - Pre-norm (input): $\mathbf{z}^{\ell+1} = f(\text{LN}(\mathbf{z}^{\ell})) + \mathbf{z}^{\ell}$ <https://arxiv.org/abs/2002.04745>
 - Pre-norm (output): $\mathbf{z}^{\ell+1} = \text{LN}(f(\mathbf{z}^{\ell})) + \mathbf{z}^{\ell}$ <https://arxiv.org/pdf/2111.09883>
 - (Does not combine well with dropout)
 - Post-norm: $\mathbf{z}^{\ell+1} = \text{LN}(f(\mathbf{z}^{\ell}) + \mathbf{z}^{\ell})$ Original transformer
- Pre-norm leads to faster training. Pre-norm (output) has less spikes.

Layer normalization

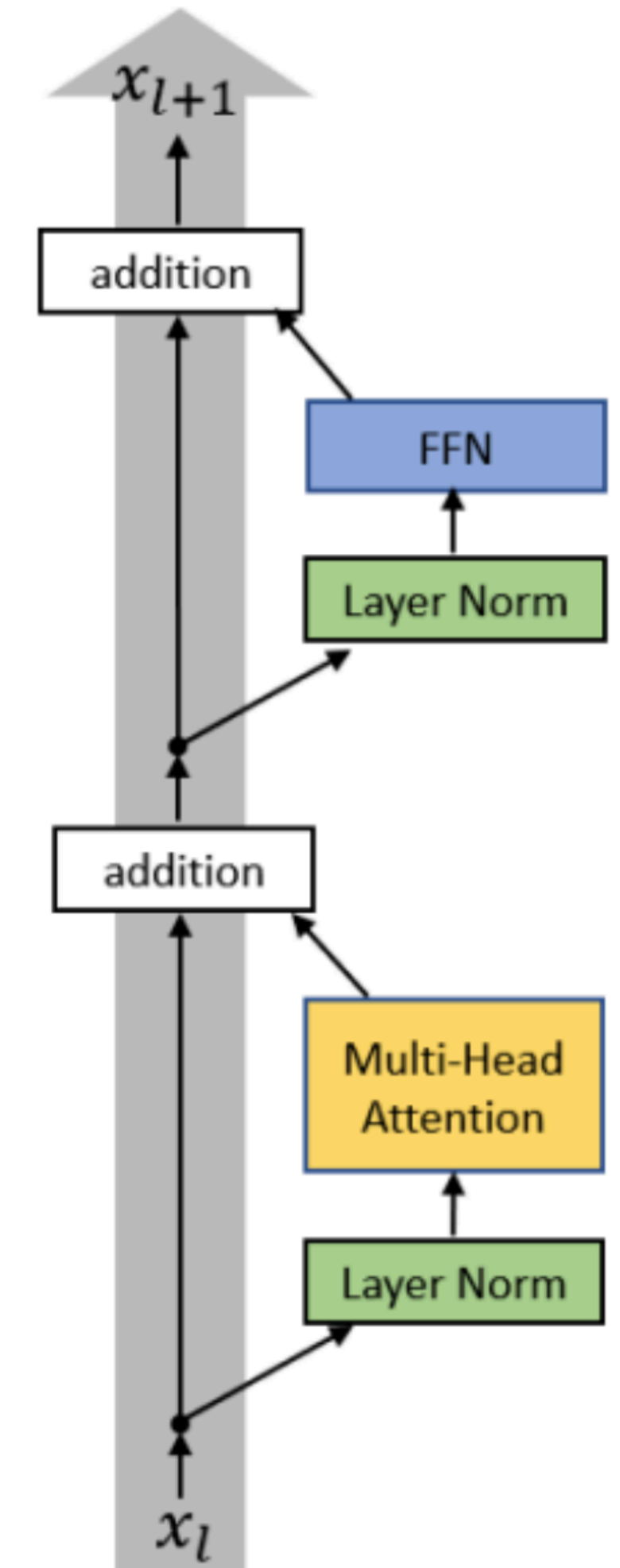
Pre- vs Post- layer normalization

- Moving layer normalization to before the MHA and FFN block helps improve training stability
- Warm-up of learning rate is critical for training when using Post-LN

Original transformer



Post

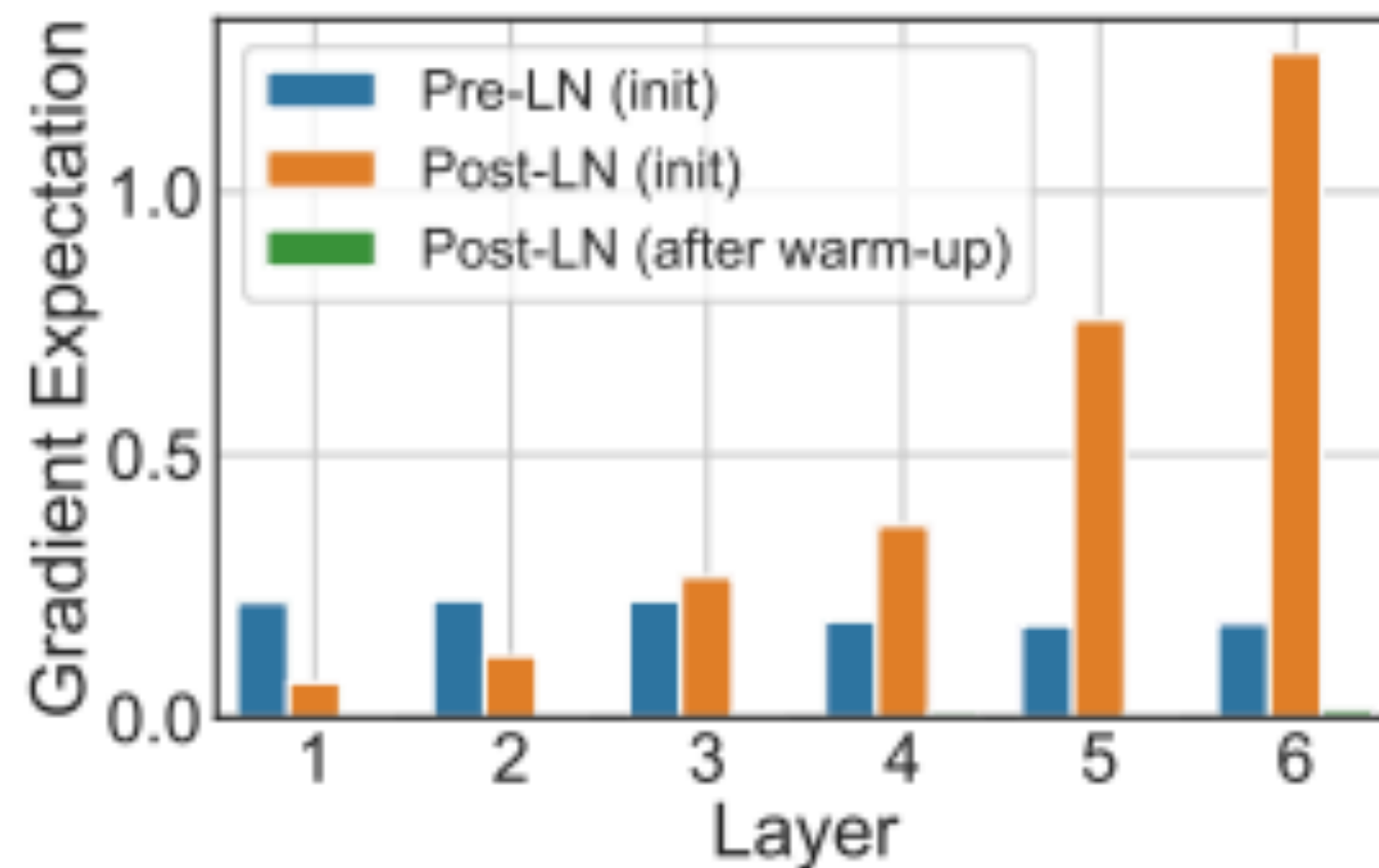


Pre

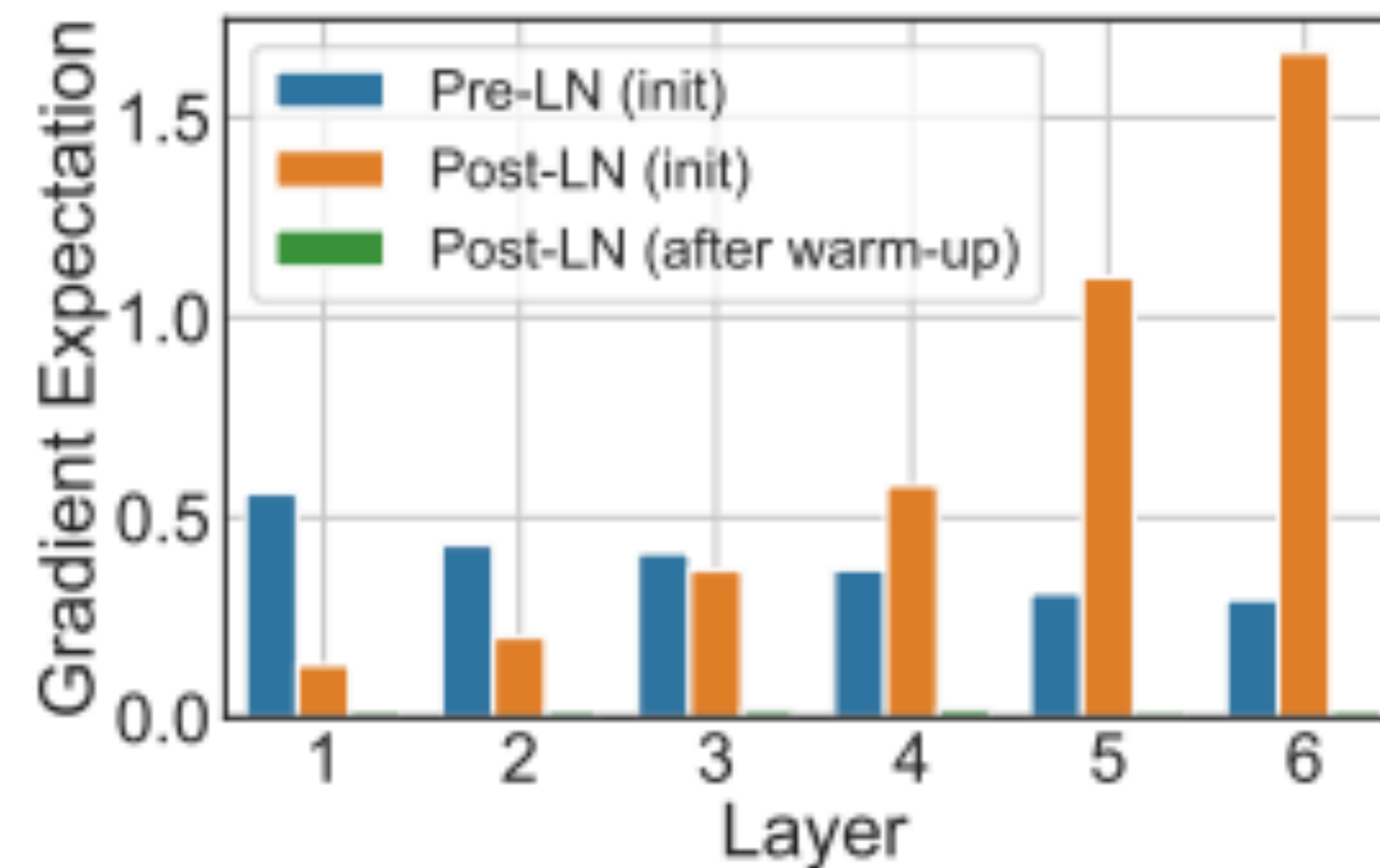
Layer normalization

Pre- vs Post- layer normalization

- Warmup very important for Post-LN
- Gradient for Post-LN varies across layers during warmup, and is very small after warmup



(a) W^1 in the FFN sub-layers



(b) W^2 in the FFN sub-layers

Replace LayerNorm with RMSNorm

LayerNorm

- changes input features to have mean 0 and variance 1 per layer.
- Adds two more learnable parameters (vectors)
 - Gain **g** and bias **b**

$$\mu^l = \frac{1}{H} \sum_{i=1}^H x_i^l \quad \sigma^l = \sqrt{\frac{1}{H} \sum_{i=1}^H (x_i^l - \mu^l)^2}$$

$$h_i = \frac{g_i}{\sigma_i} (x_i - \mu_i) + b_i$$

RMSNorm

- Simplifies LayerNorm by removing the mean and bias terms

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}$$

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \cdot \mathbf{g}$$

Root Mean Square Layer Normalization
[Zhang and Sennrich, 2019]

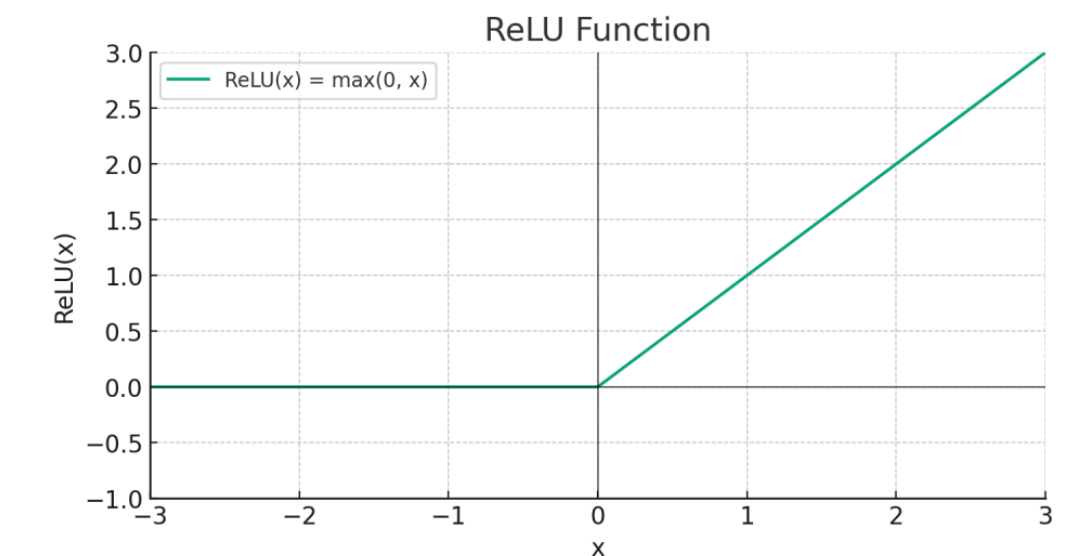
Revisiting transformers

- Positional encoding
- Normalization
 - Layer normalization vs RMS
 - Post vs Pre-Layer norm
- **Activation functions**
- Attention variations

Activation function

- Vaswani et al.: ReLU

$$\text{ReLU}(\mathbf{x}) = \max(0, \mathbf{x})$$



Gated linear unit (GLU)

- Output of a linear transformation of $\mathbf{x}$ is modulated by a gate

$$\text{GLU} = \sigma(xW) \otimes xV$$

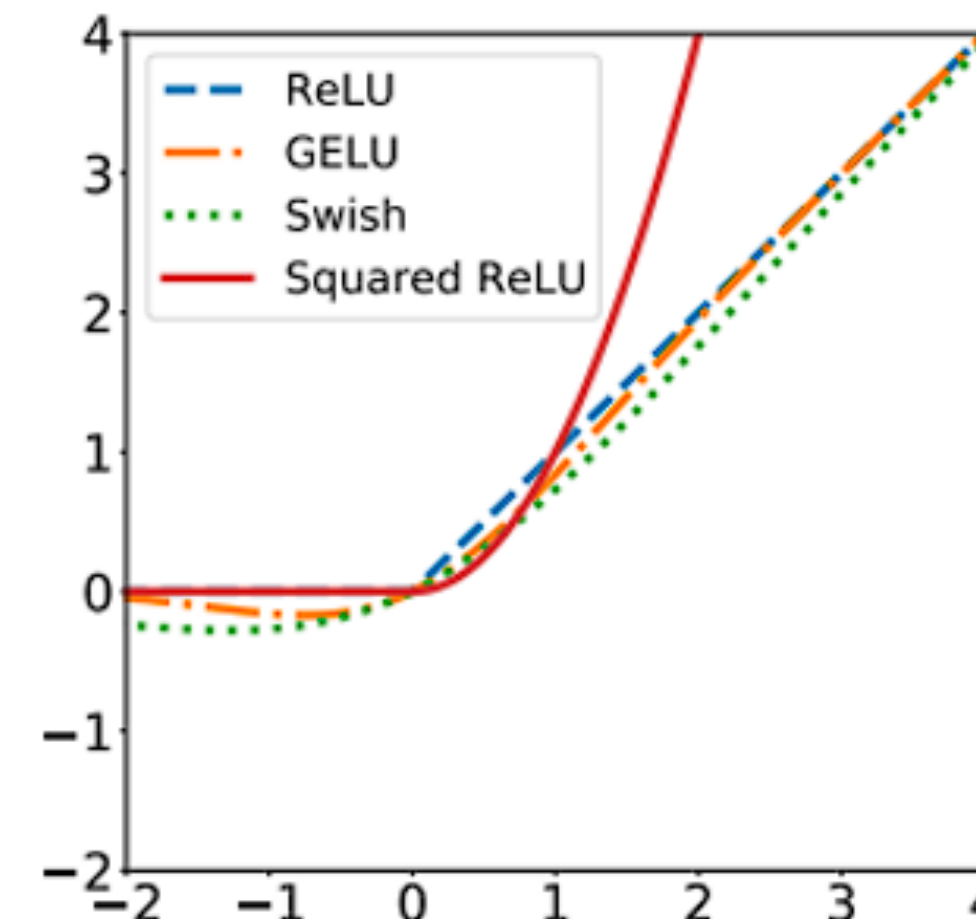
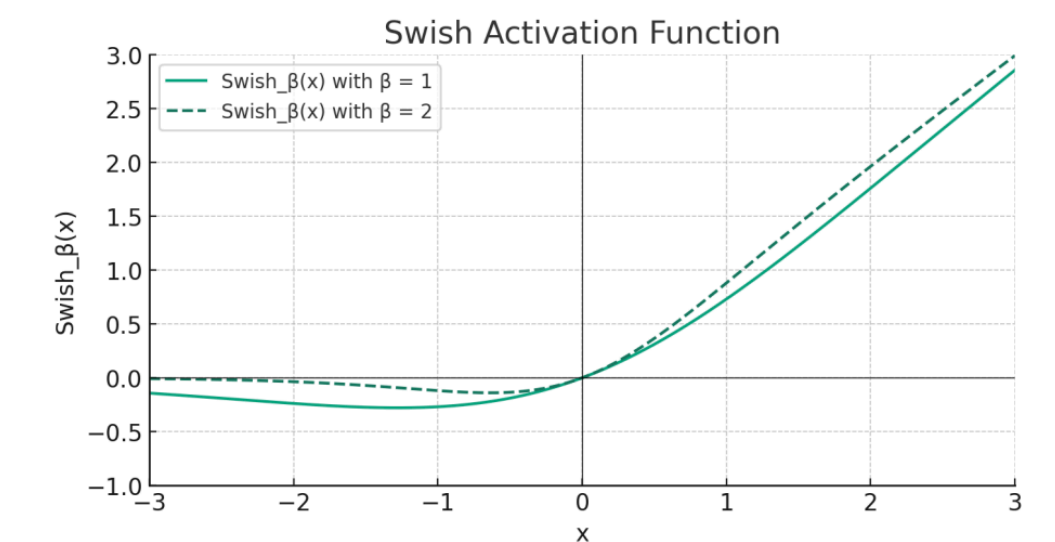
$$\text{SwiGLU} = \text{Swish}(xW) \otimes xV$$

Squared ReLU (GLU)

- Outperforms ReLU, competitive with SwiGLU
- Sparse computations
- Used in Primer

- LLaMa: Swish/SiLU (Hendricks and Gimpel 2016)

$$\text{Swish}(\mathbf{x}; \beta) = \mathbf{x} \odot \sigma(\beta \mathbf{x})$$



Revisiting transformers

- Positional encoding
- Normalization
 - Layer normalization vs RMS
 - Post vs Pre-Layer norm
- Activation functions
- **Attention variations**

QK norm

- Normalize query and key to unit vectors

$$A(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \longrightarrow$$

γ is learnable parameter

$$A(Q, K, V) = \text{softmax} (\gamma \cdot Q' K'^T) V$$
$$Q'_i = \frac{Q_i}{\|Q_i\|}, K'_j = \frac{K_j}{\|K_j\|}$$

- Variants using LayerNorm
- Training stability

Parallel attention and QK norm

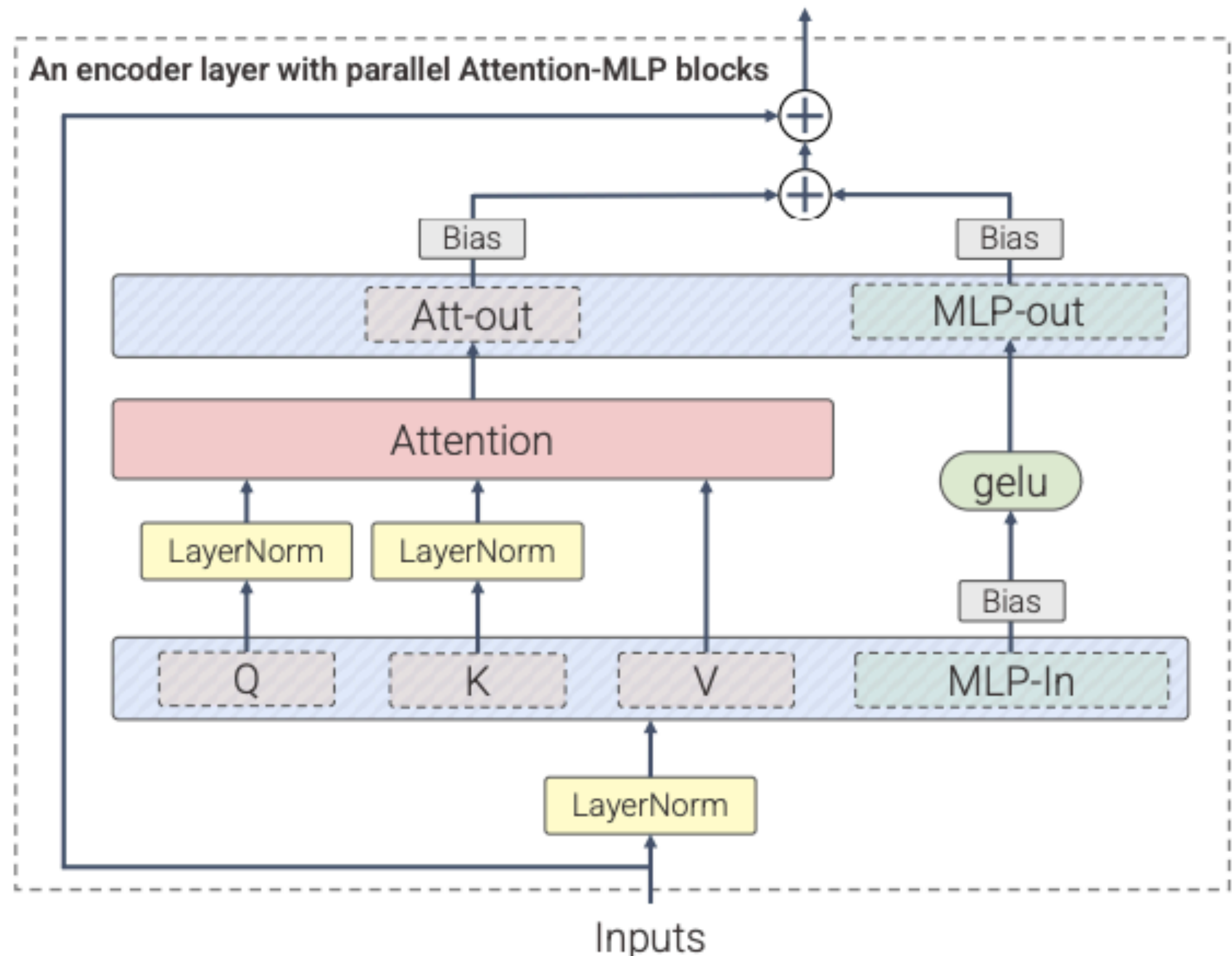
- Parallel attention

$$y' = \text{LayerNorm}(x),$$

$$y = x + \text{MLP}(y') + \text{Attention}(y').$$

- QK norm

$$\text{softmax} \left[\frac{1}{\sqrt{d}} \text{LN}(XW^Q) (\text{LN}(XW^K))^T \right]$$



QK norm

- Prevents exploding attention logits (which leads to almost one-hot attention weights with near-zero entropy)

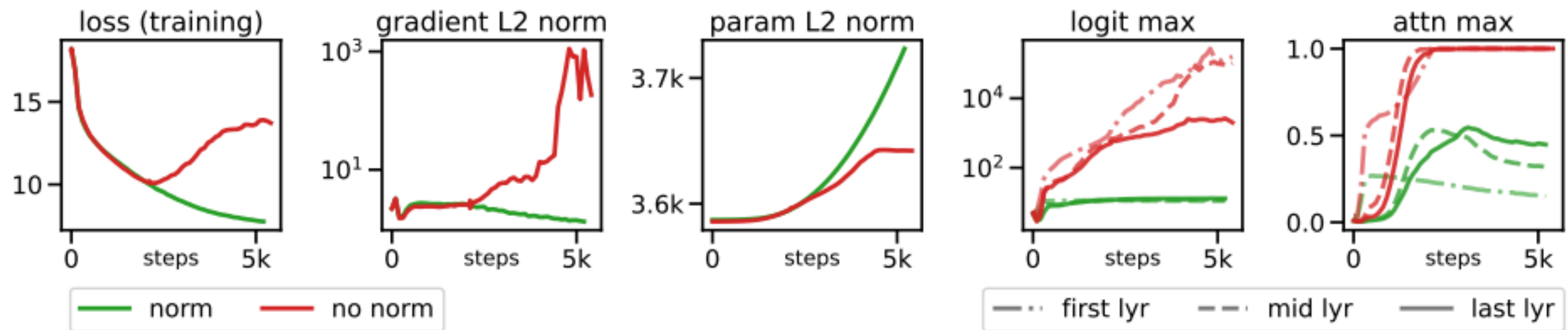
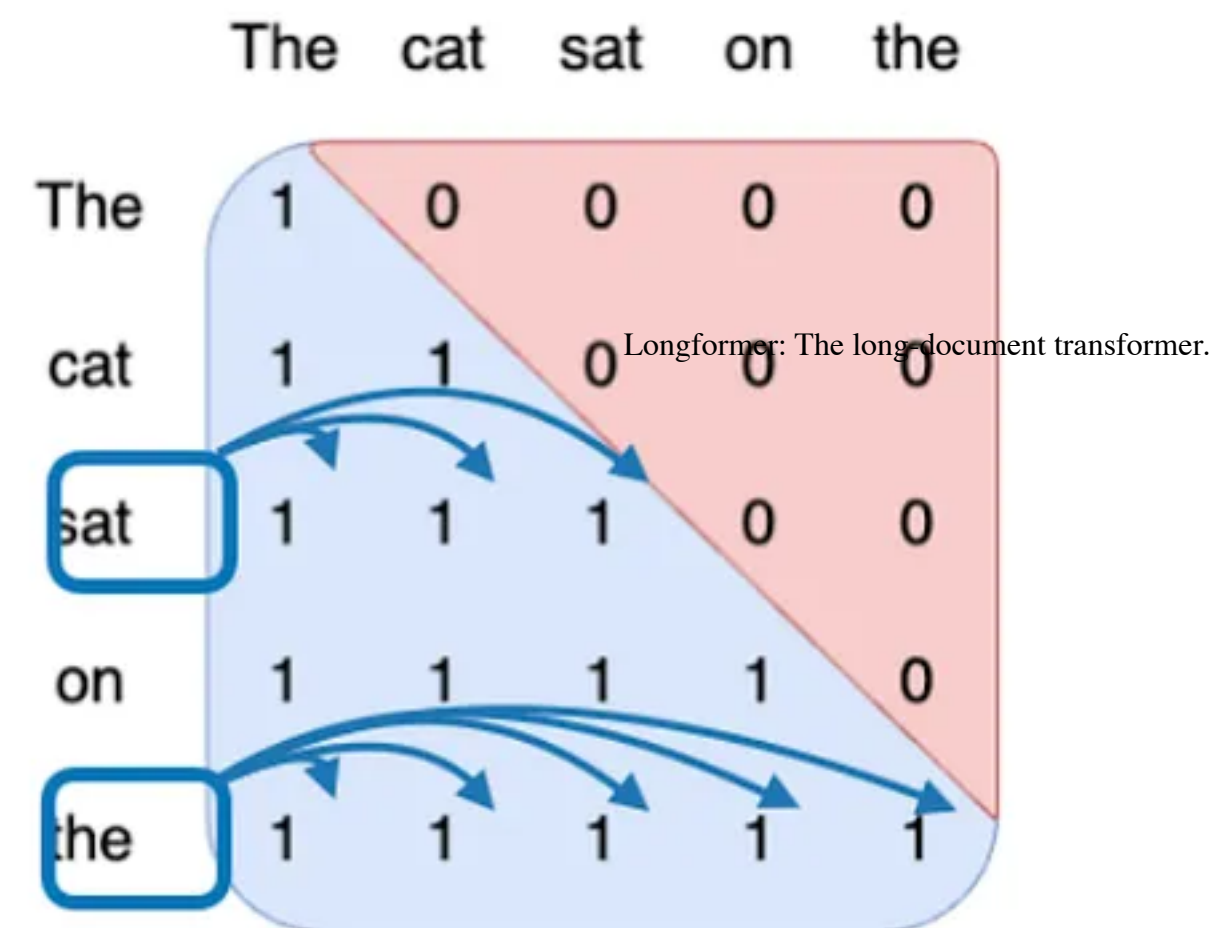


Figure 1: Effect of query/key normalization on an 8B parameter model.

Sliding window attention

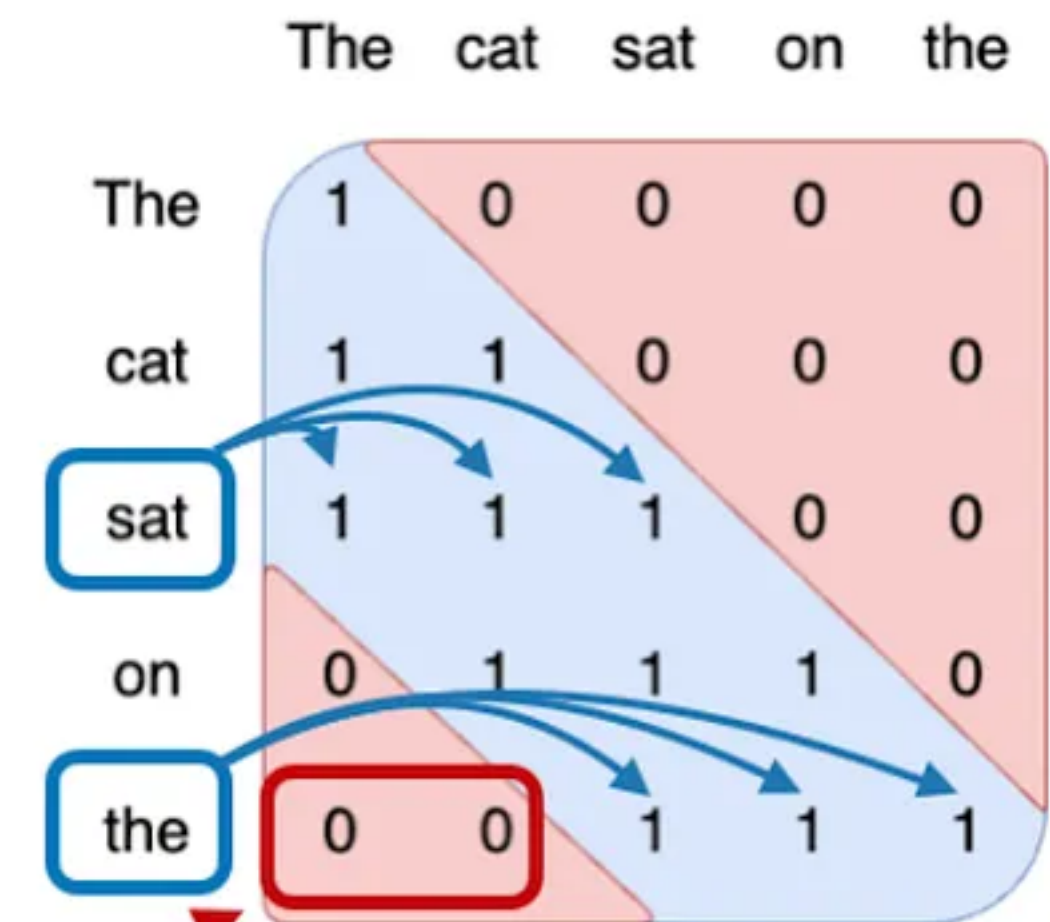
- Limited context / local attention
- Interleave with global attention layers

Regular causal self-attention mask



Using a causal attention mask, the current token can only attend previous tokens (+ itself)

Sliding window attention

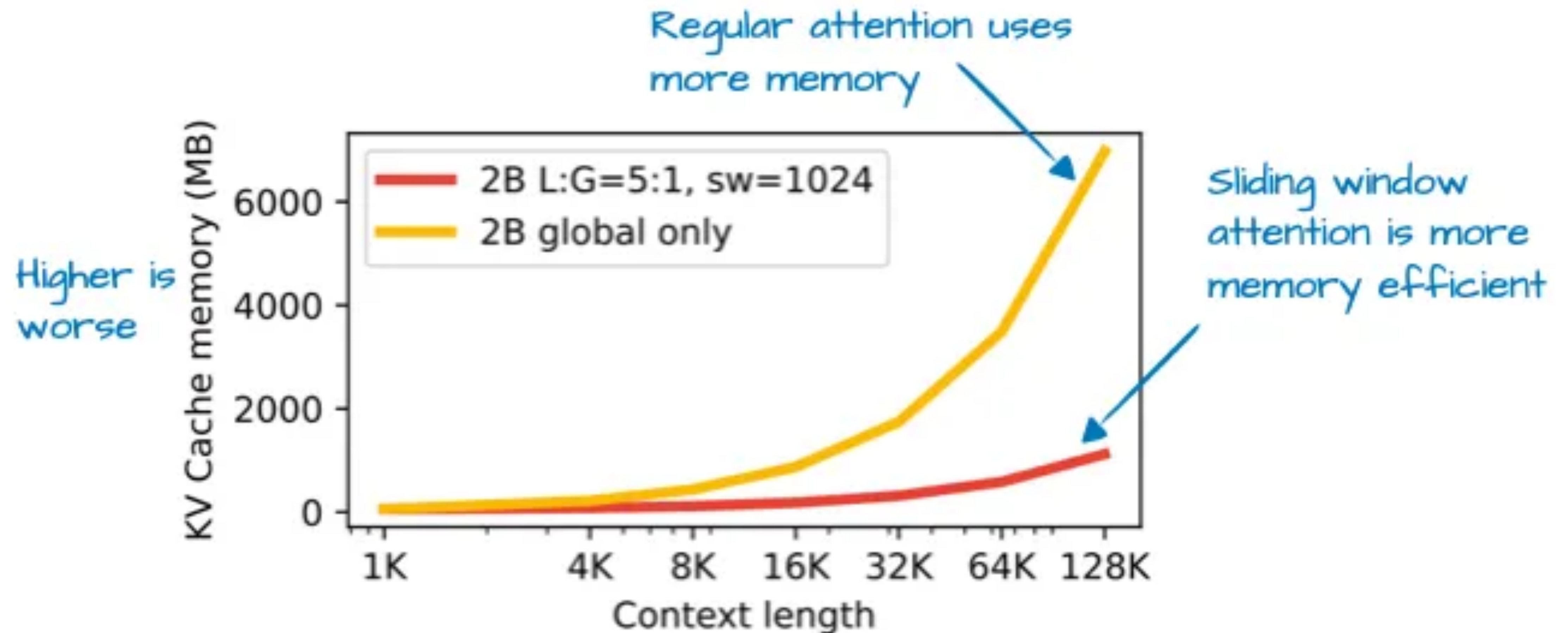


Not attended to save computation

Using a causal attention mask, the current token can only attend previous tokens **within a certain limit**

<https://magazine.sebastianraschka.com/p/the-big-llm-architecture-comparison>

Sliding window attention

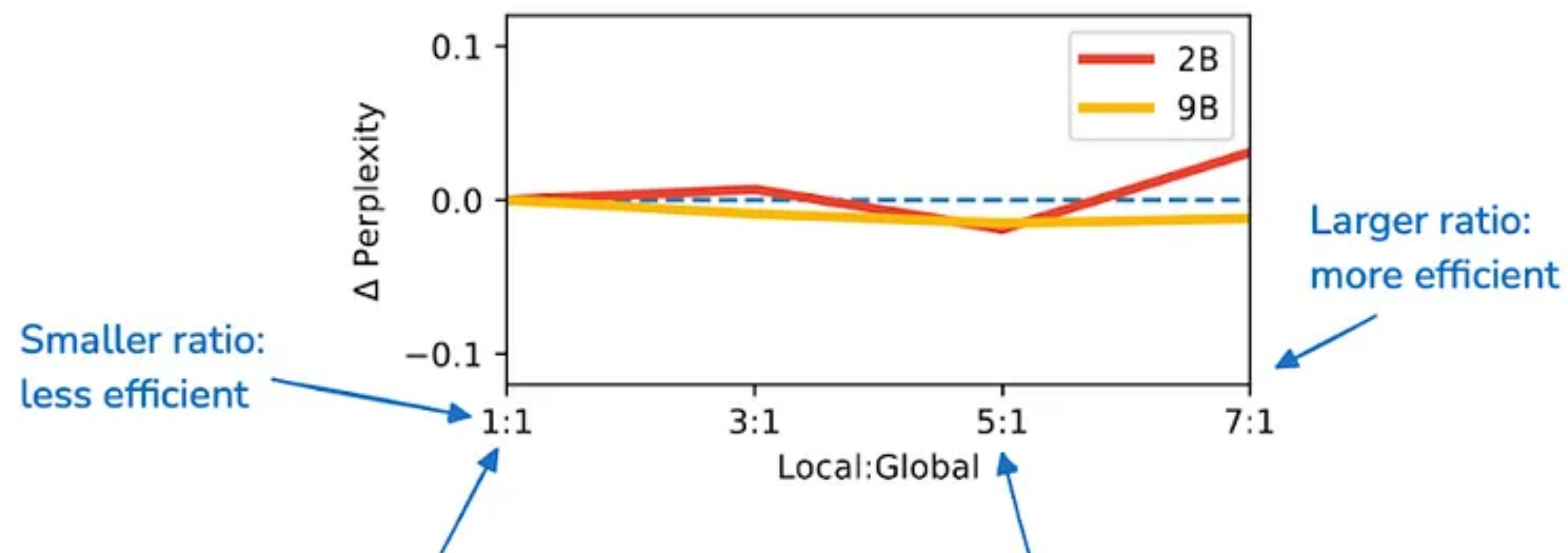


Gemma 3 [Gemma Team. 2025]

(Annotated version from <https://magazine.sebastianraschka.com/p/the-big-llm-architecture-comparison>)

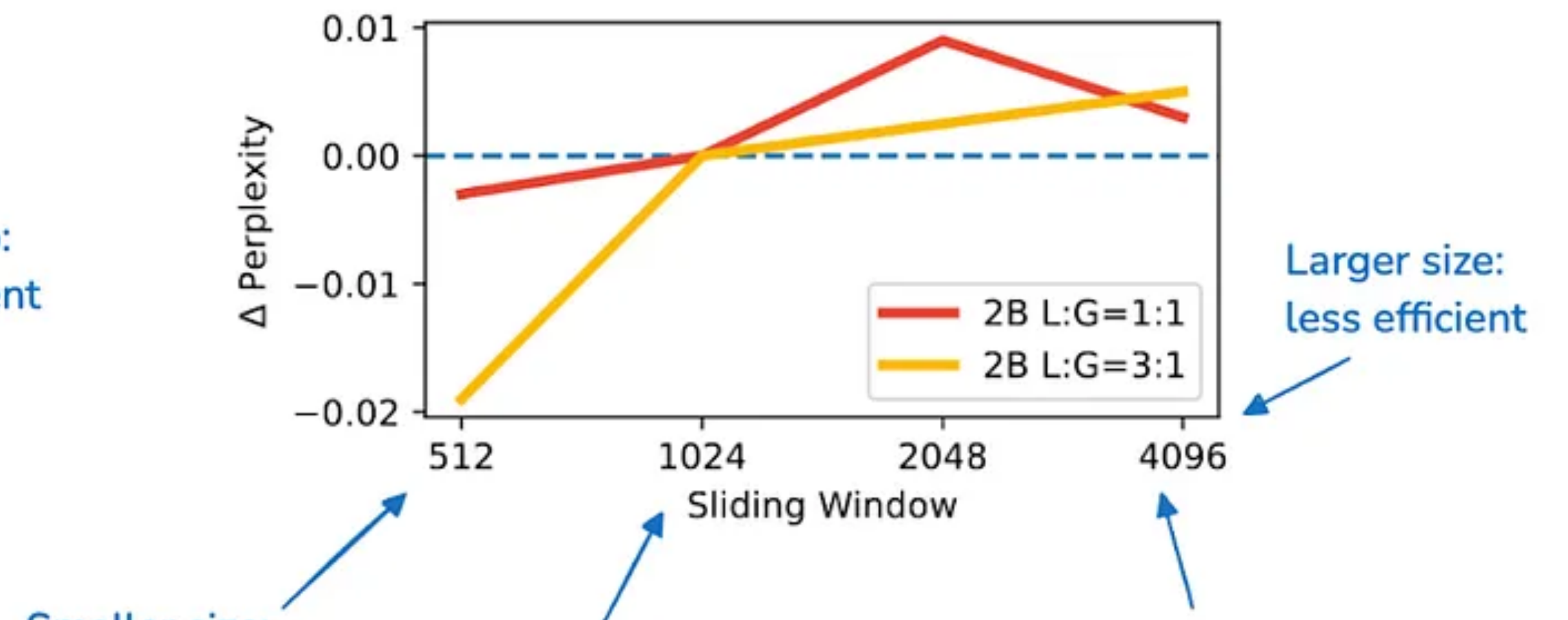
Sliding window attention

Full (regular) vs sliding window attention



Whether to use a 1:1 (Gemma 2) or 5:1 (Gemma 3) ratio of full (global) and sliding window (local) attention layers has no significant impact on the modeling performance

Sliding window size



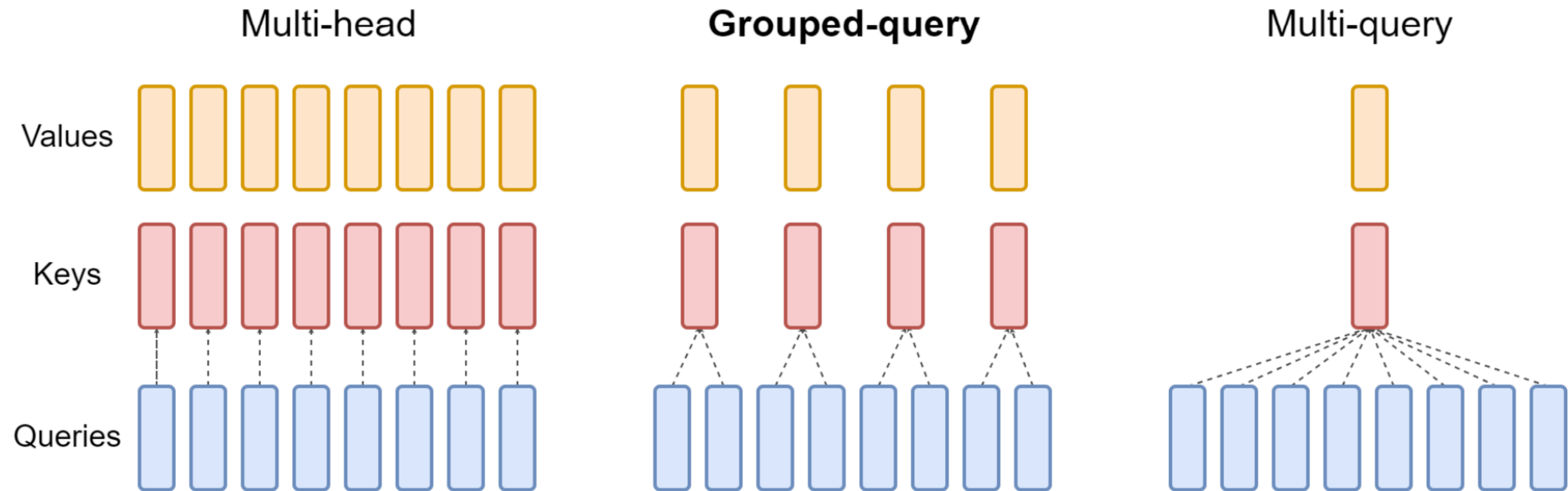
Negligible modeling performance difference whether a sliding window size of 1024 (Gemma 3) or 4096 (Gemma 2) is used.

Gemma 3 [Gemma Team. 2025]

(Annotated version from <https://magazine.sebastianraschka.com/p/the-big-llm-architecture-comparison>)

Grouped multi-query attention

- Reduce number of heads used for keys and values
- Shared values and keys across heads



Fast Transformer Decoding: One Write-Head is All You Need [Shazeer 2019]

GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints [Ainslie et al. 2023]

Multi-head latent attention (MLA)

- Use low rank matrix C^{KV}
- Project from latent space to full KV

$$\begin{aligned} C_n^{KV} &= W^{DKV} x_n \\ K &= W^{UK} C_{1:n}^{KV} \\ V &= W^{UV} C_{1:n}^{KV} \end{aligned}$$

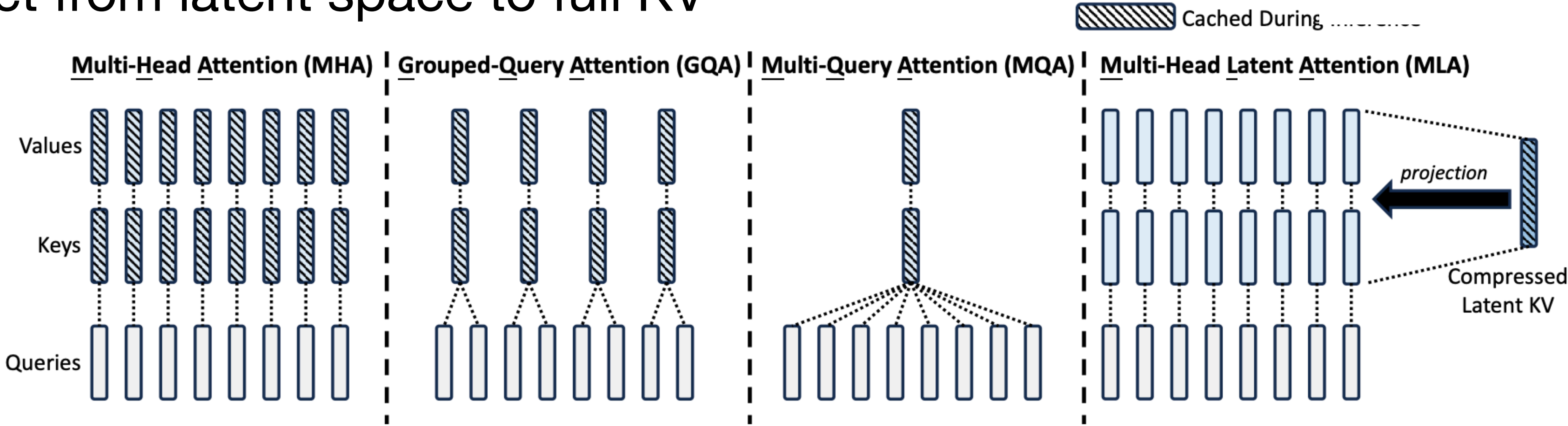
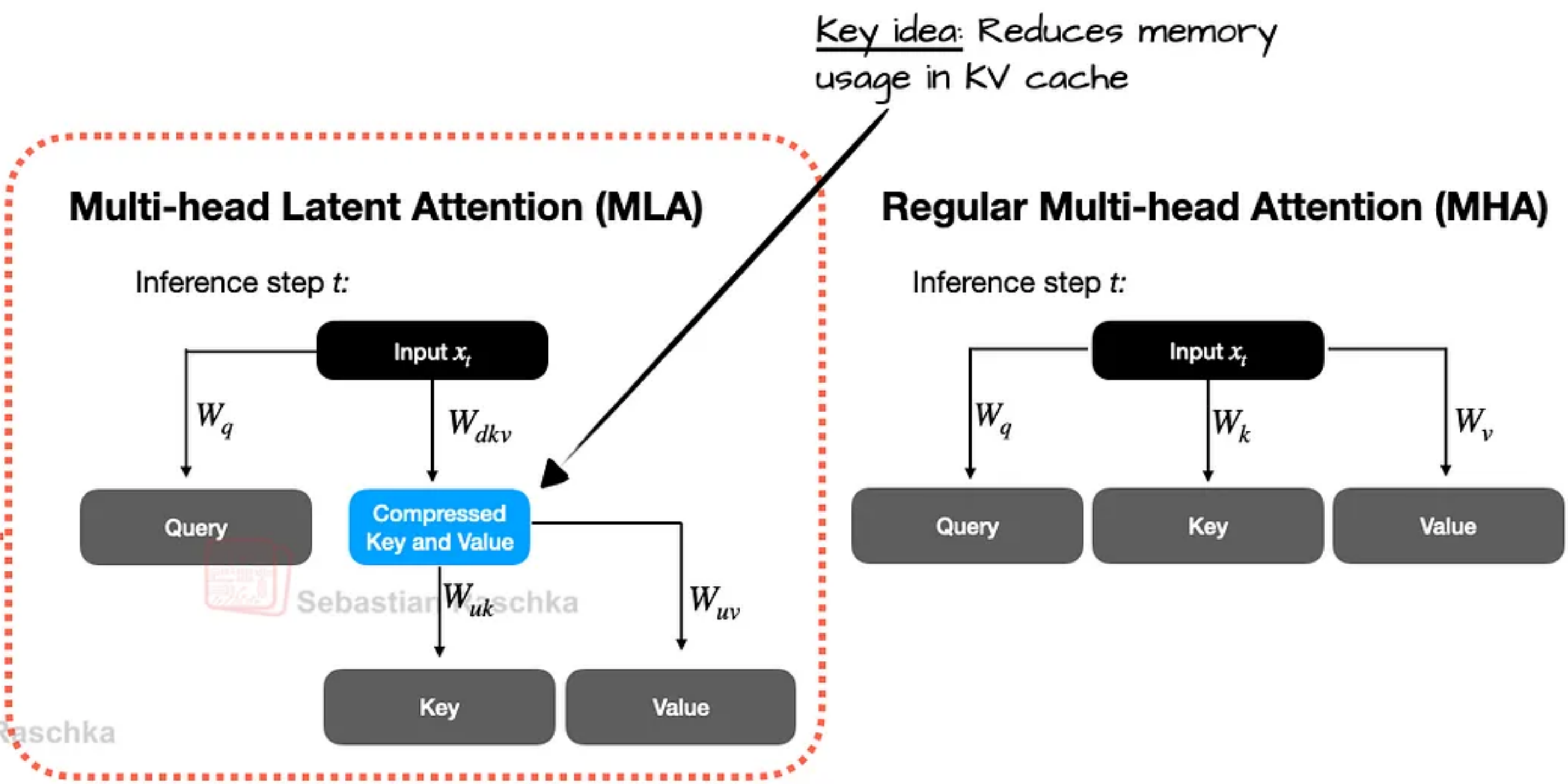
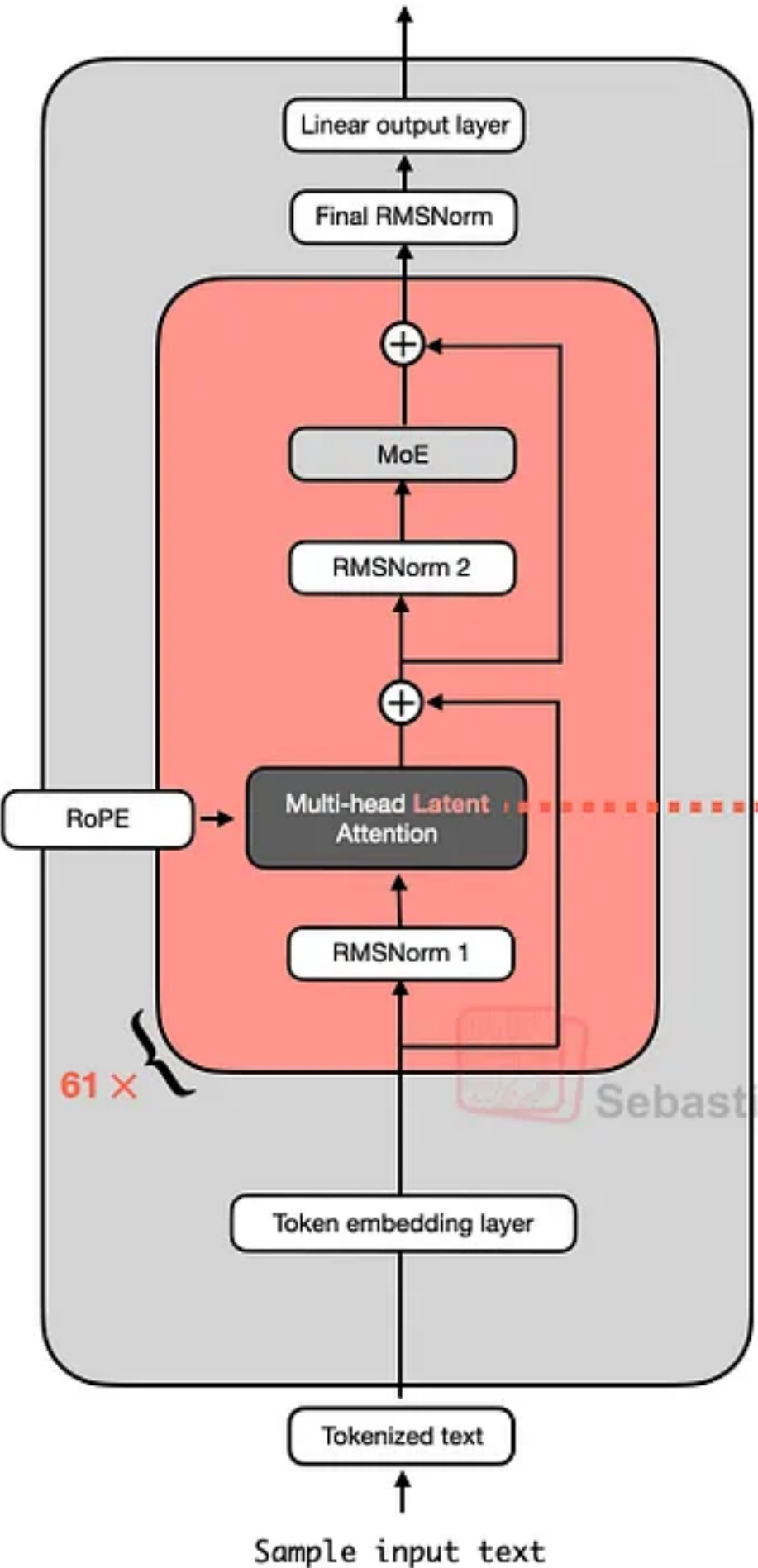


Figure 3 | Simplified illustration of Multi-Head Attention (MHA), Grouped-Query Attention (GQA), Multi-Query Attention (MQA), and Multi-head Latent Attention (MLA). Through jointly compressing the keys and values into a latent vector, MLA significantly reduces the KV cache during inference.

Multi-head latent attention (MLA)

DeepSeek V3/R1



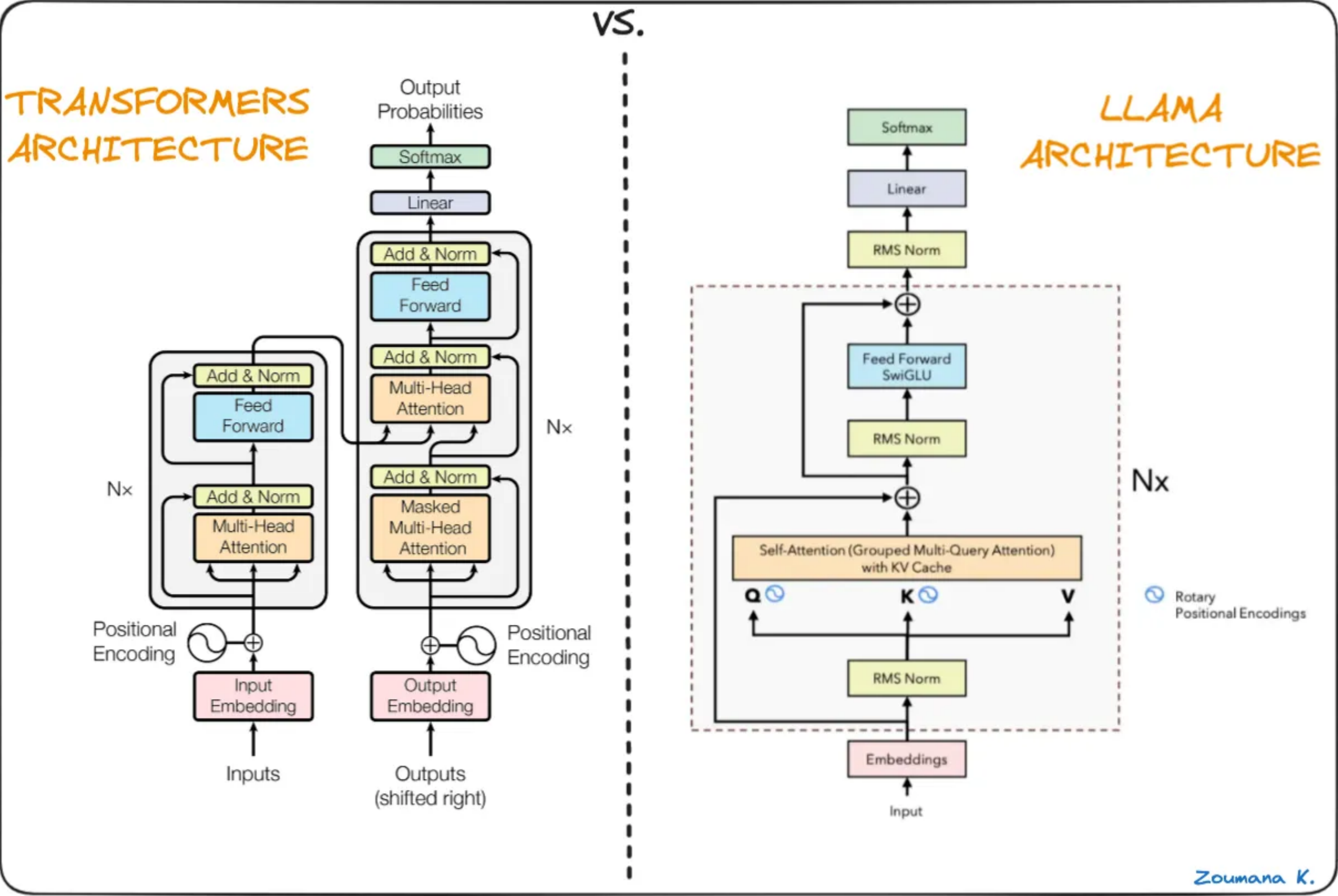
Gated value embedding

- Add gated value embedding (at alternating layers)
- Additional expressivity (more parameters) with minimal compute overhead

```
ve = value_embeds[layer_idx](token_ids) # (B, T, kv_dim)
gate = 2 * sigmoid(ve_gate(x[:, :, :32])) # range (0, 2)
v = v + gate * ve
```

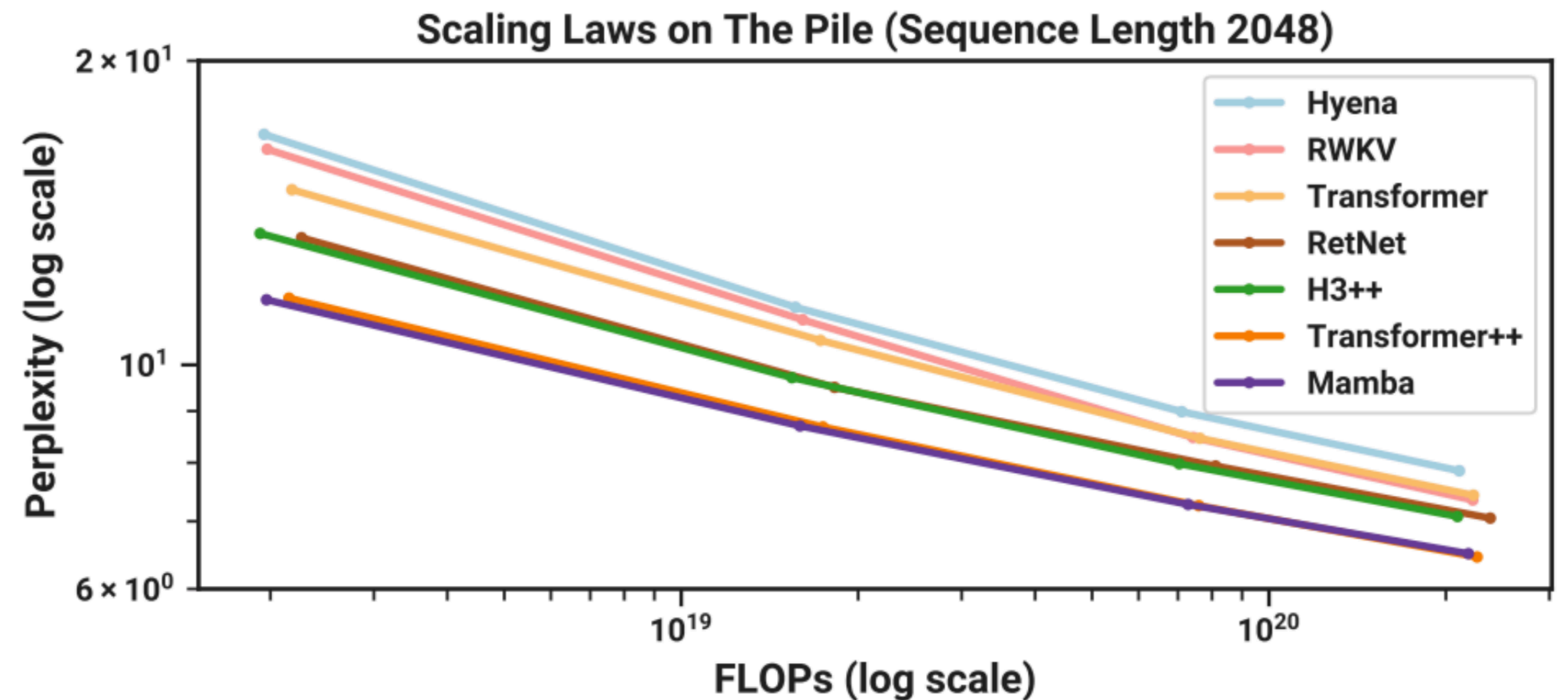

LLaMa

	Vaswani et al.	LLaMa
Norm Position	Post	Pre
Norm Type	LayerNorm	RMSNorm
Non-linearity	ReLU	SwiGLU
Positional Encoding	Sinoidal	RoPE
Attention	Full Multi-Head Attention	Grouped Multi-Query Attention



Impact of these changes

- Transformer: Vaswani et al.
- Transformer++: basically LLaMa

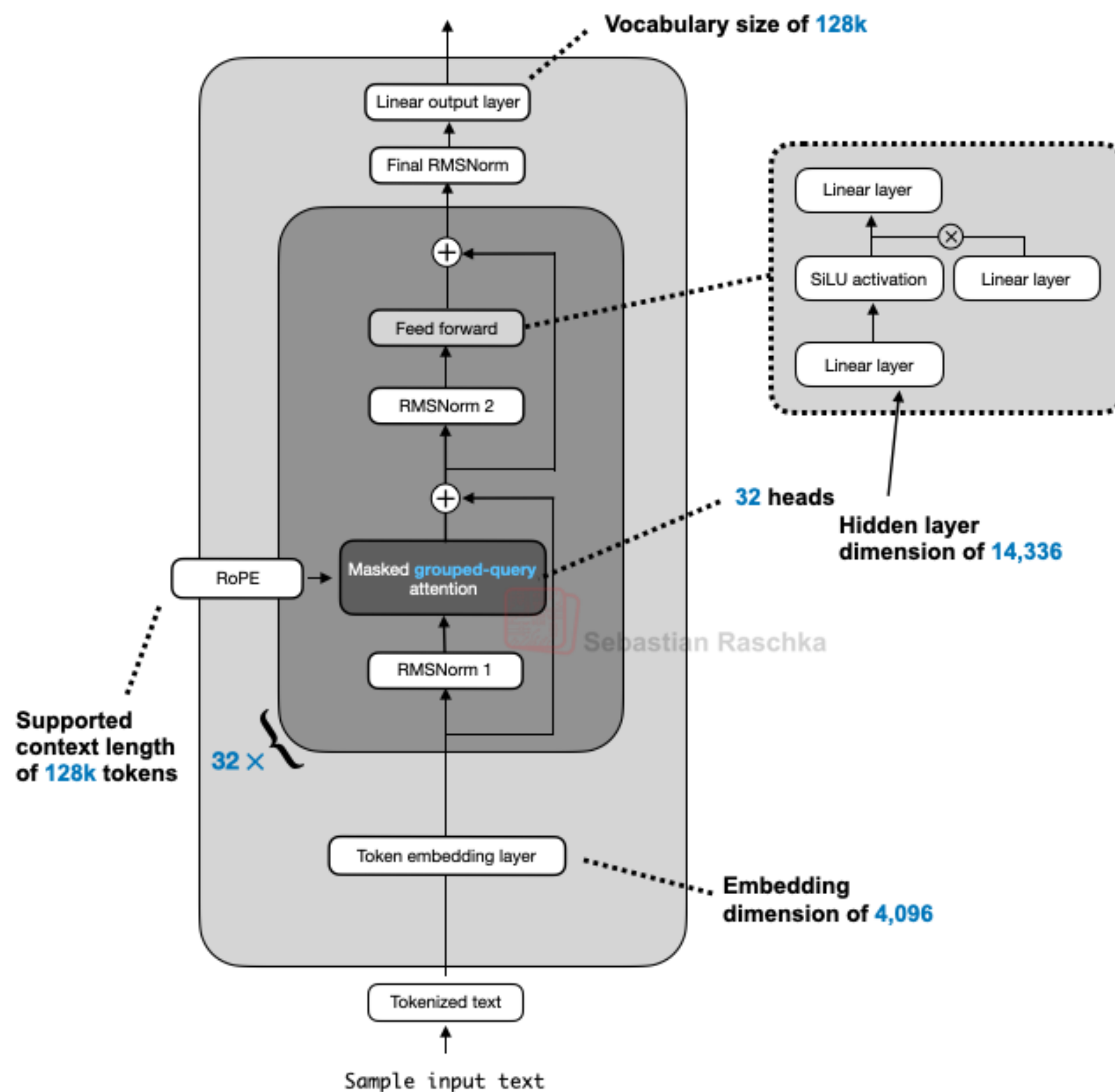


- Stronger architecture is $\approx 10x$ more efficient!

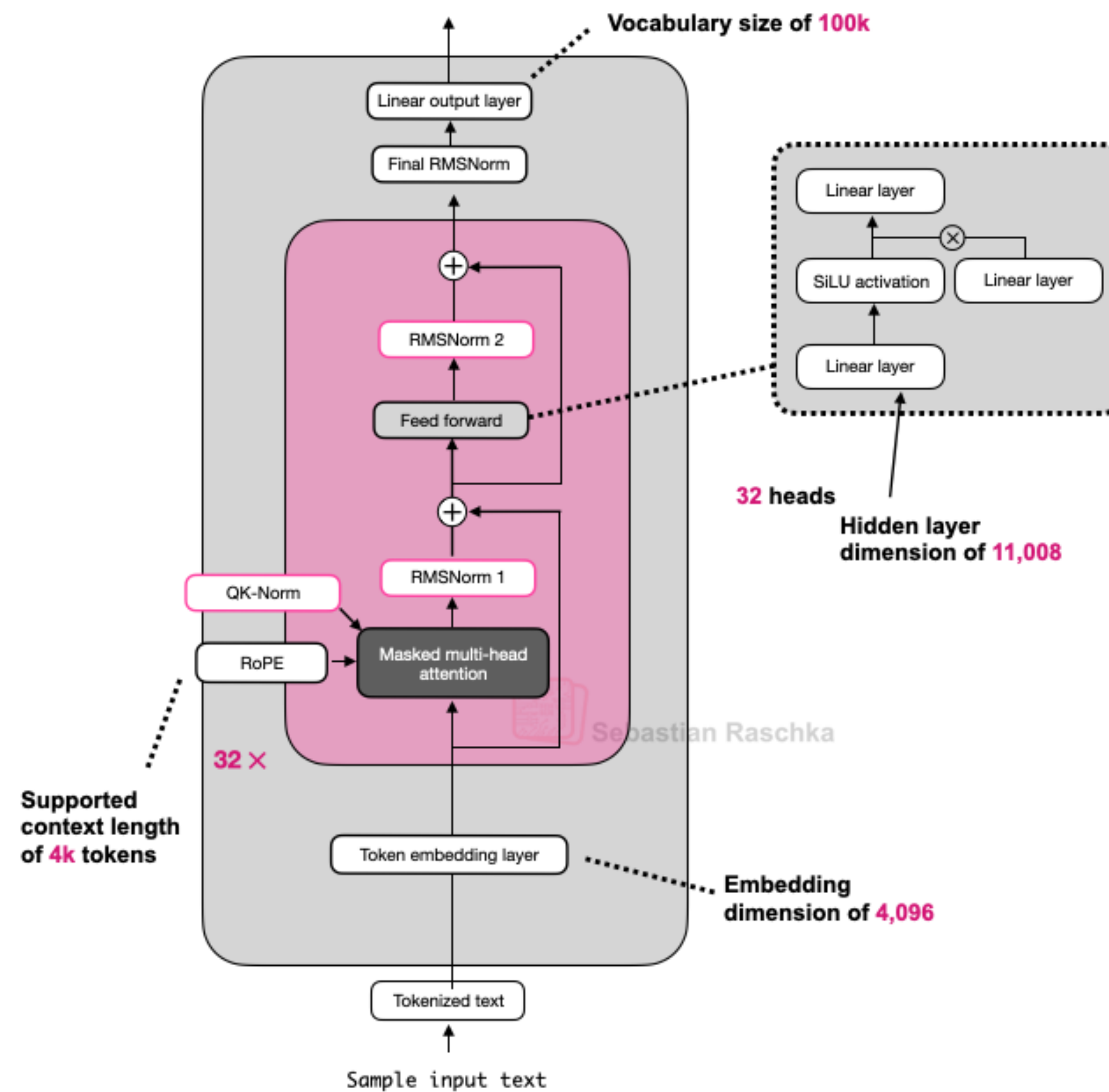
OLMo 2 architecture

	OLMo 1 (0224)	OLMo-0424	OLMo 2
Biases	None	None	None
Activation	SwiGLU	SwiGLU	SwiGLU
RoPE θ	$1 \cdot 10^4$	$1 \cdot 10^4$	$5 \cdot 10^5$
QKV Normalization	None	Clip to 8	QK-Norm
Layer Norm	non-parametric	non-parametric	RMSNorm
Layer Norm Applied to	Inputs	Inputs	Outputs
Z-Loss Weight	0	0	10^{-5}
Weight Decay on Embeddings	Yes	Yes	No

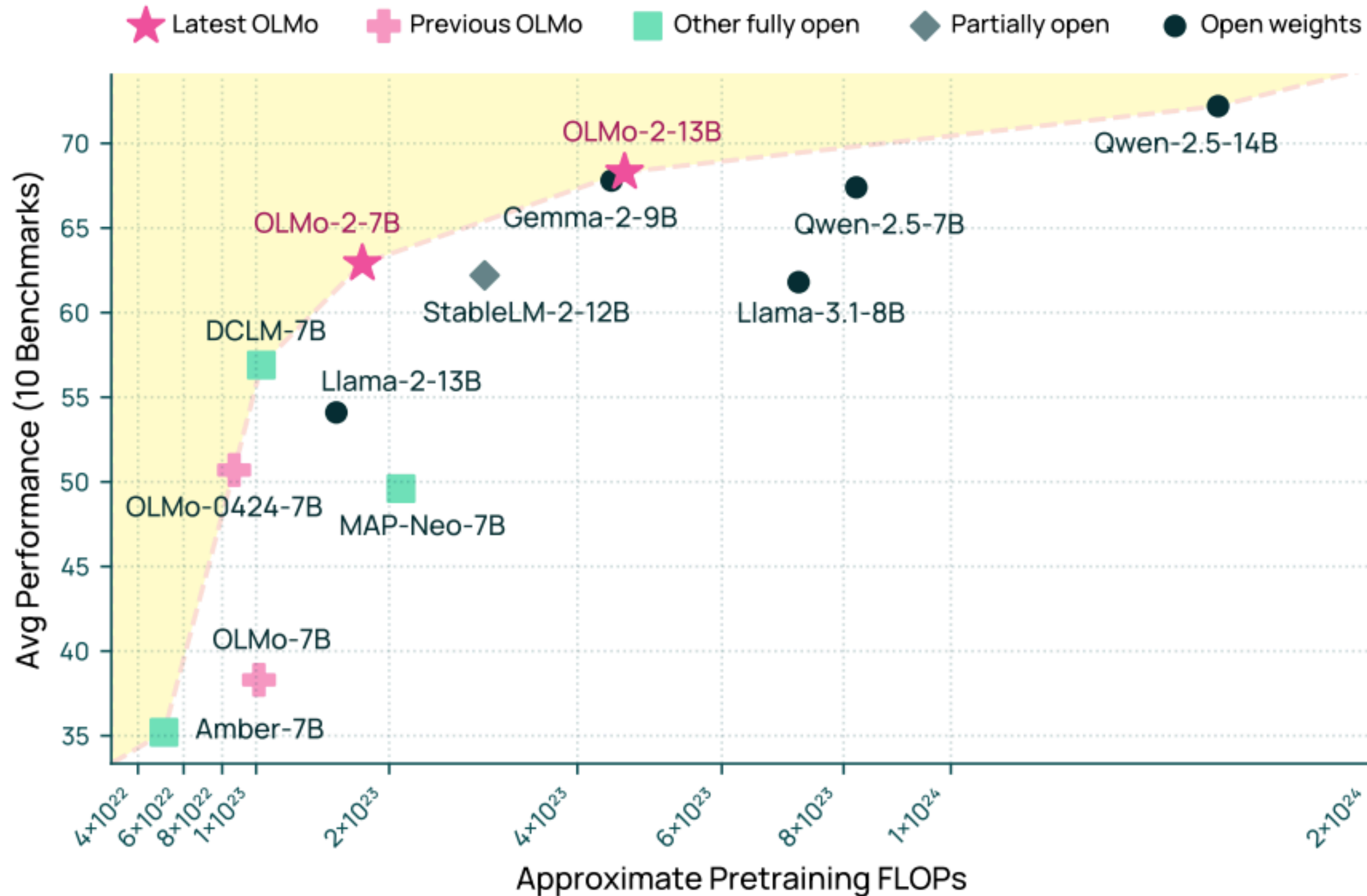
Llama 3 8B



OLMo 2 7B



Development of Open LLMs

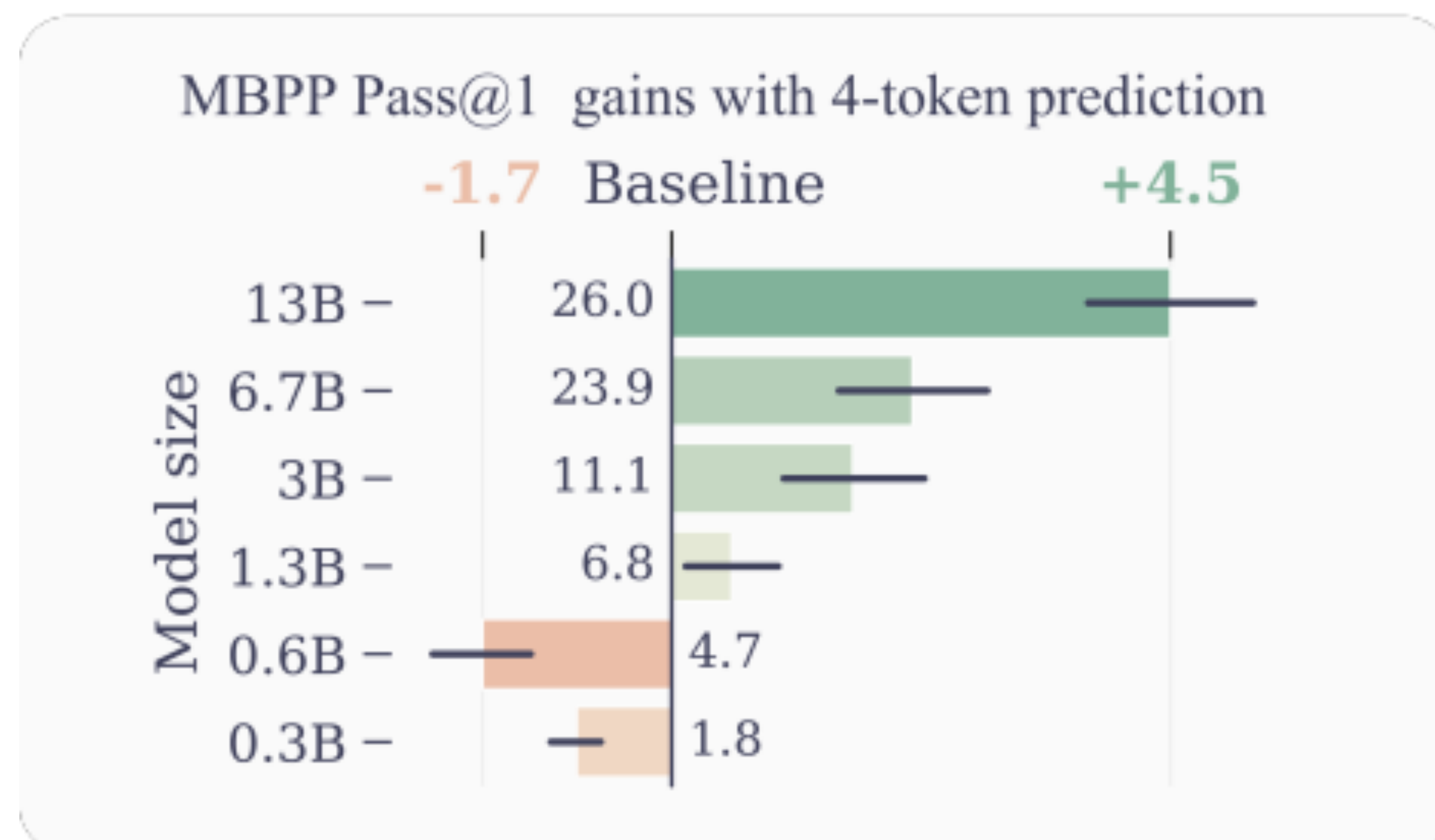
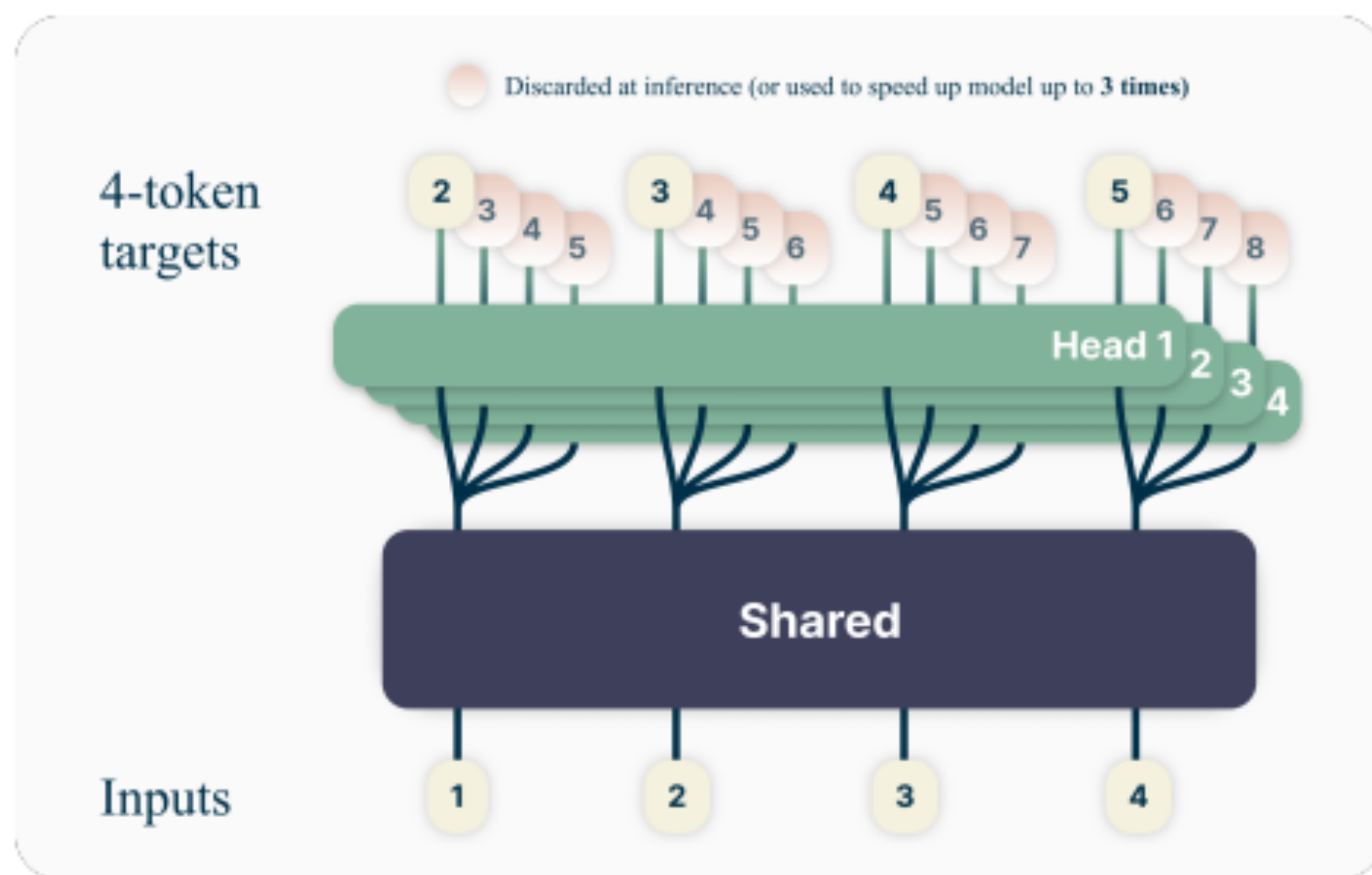


Other aspects of variation

- Training objective
- Scaling up with Mixture-of-Experts
- Optimizer
- Efficient training

Multi-token prediction

- Predict next tokens with shared trunk
- Gains due to multi-token prediction



Multi-token prediction

- Predict in sequence to avoid large memory demands

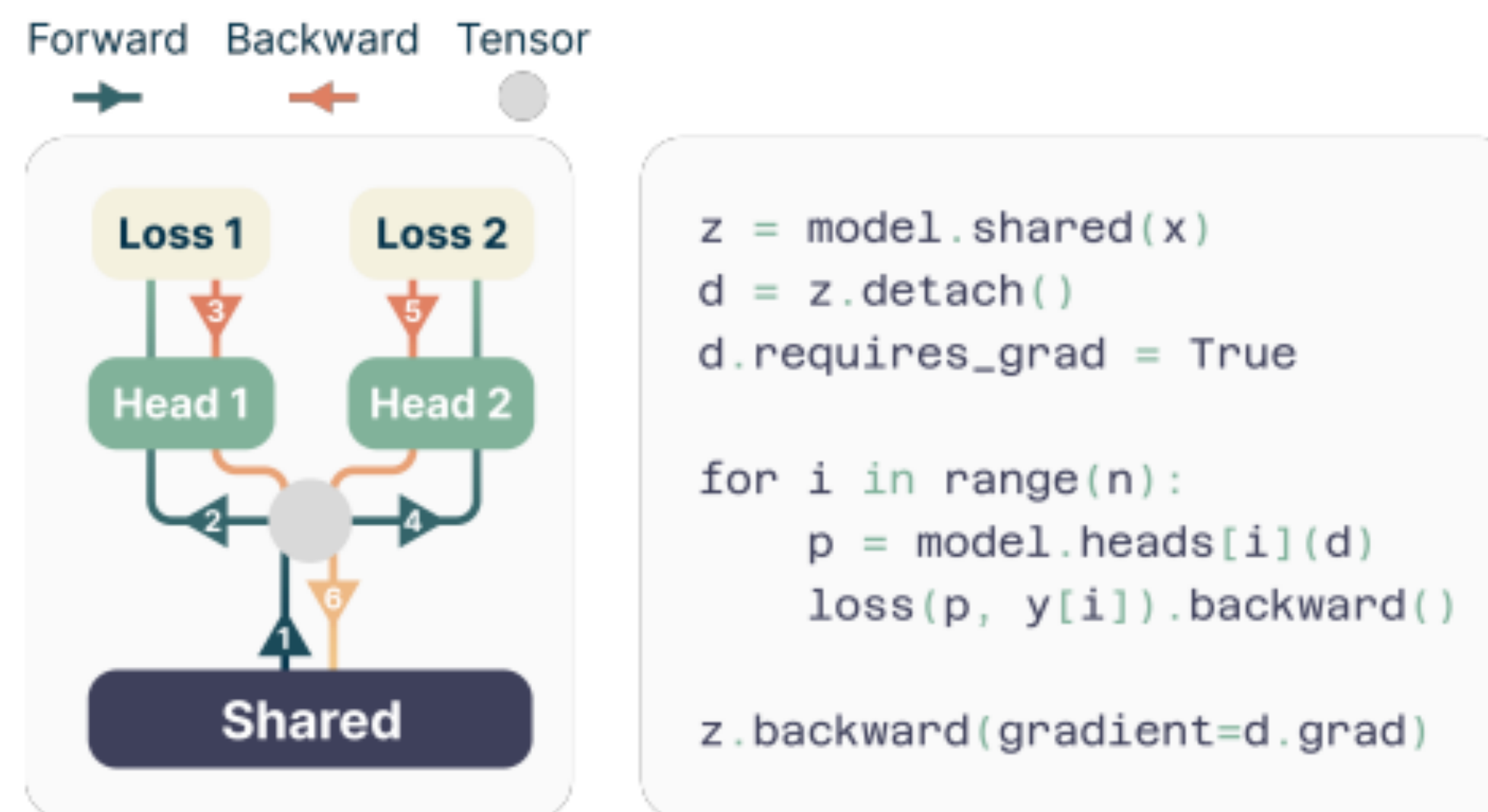
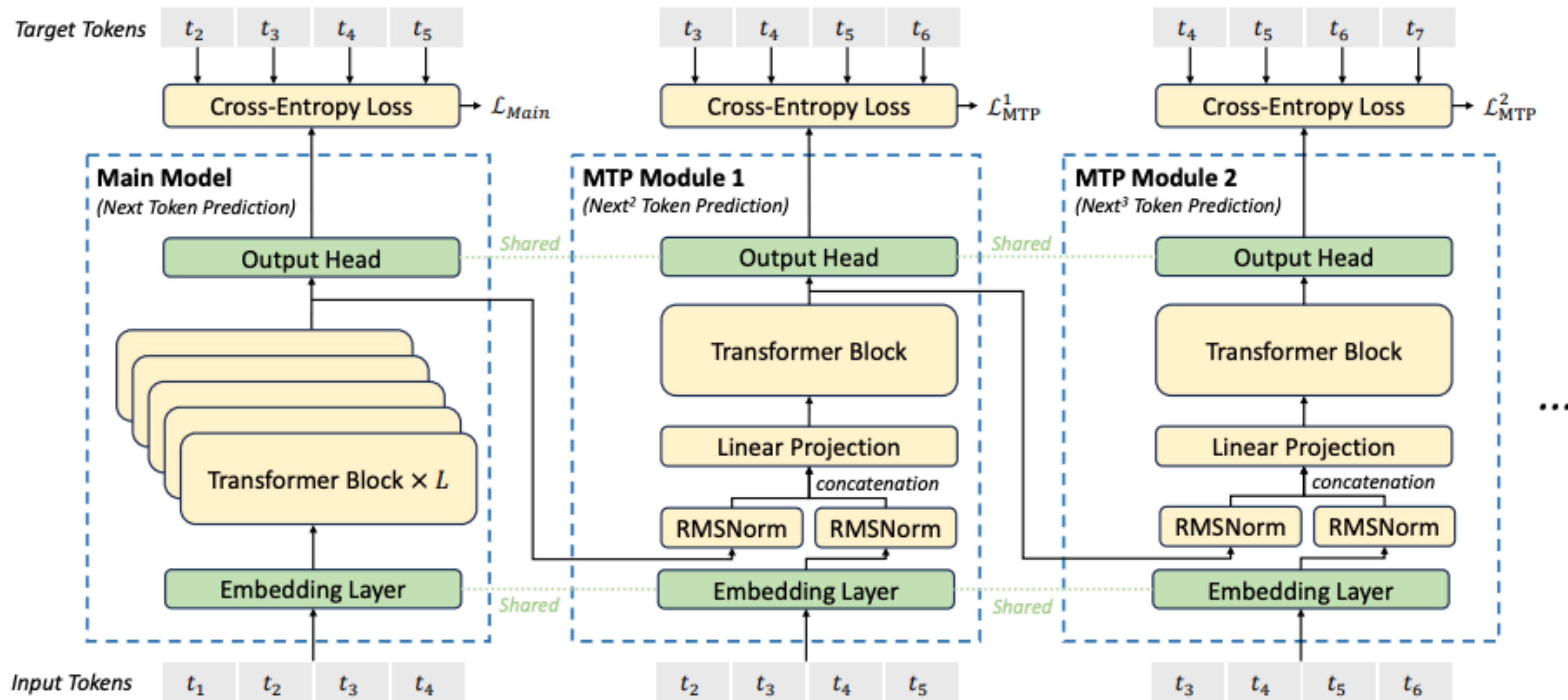


Figure 2: **Order of the forward/backward in an n -token prediction model with $n = 2$ heads.** By performing the forward/backward on the heads in sequential order, we avoid materializing all unembedding layer gradients in memory simultaneously and reduce peak GPU memory usage.

Multi-token prediction

- D sequential modules for predicting D tokens



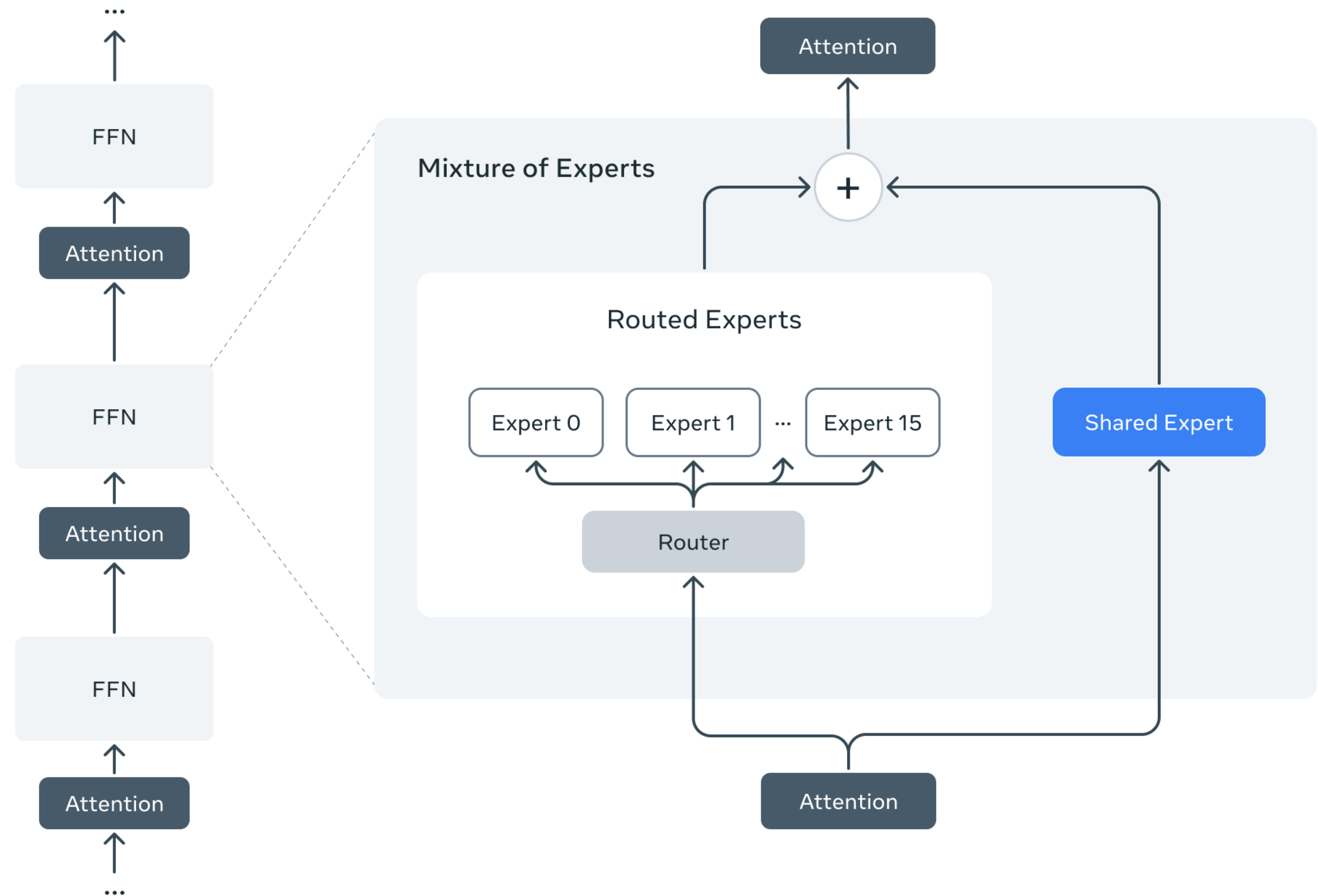
LLama 4 - 17B active parameters, 400B total parameters

Alternating dense and MoE layers

MOE layer: 128 routed expert, 1 shared

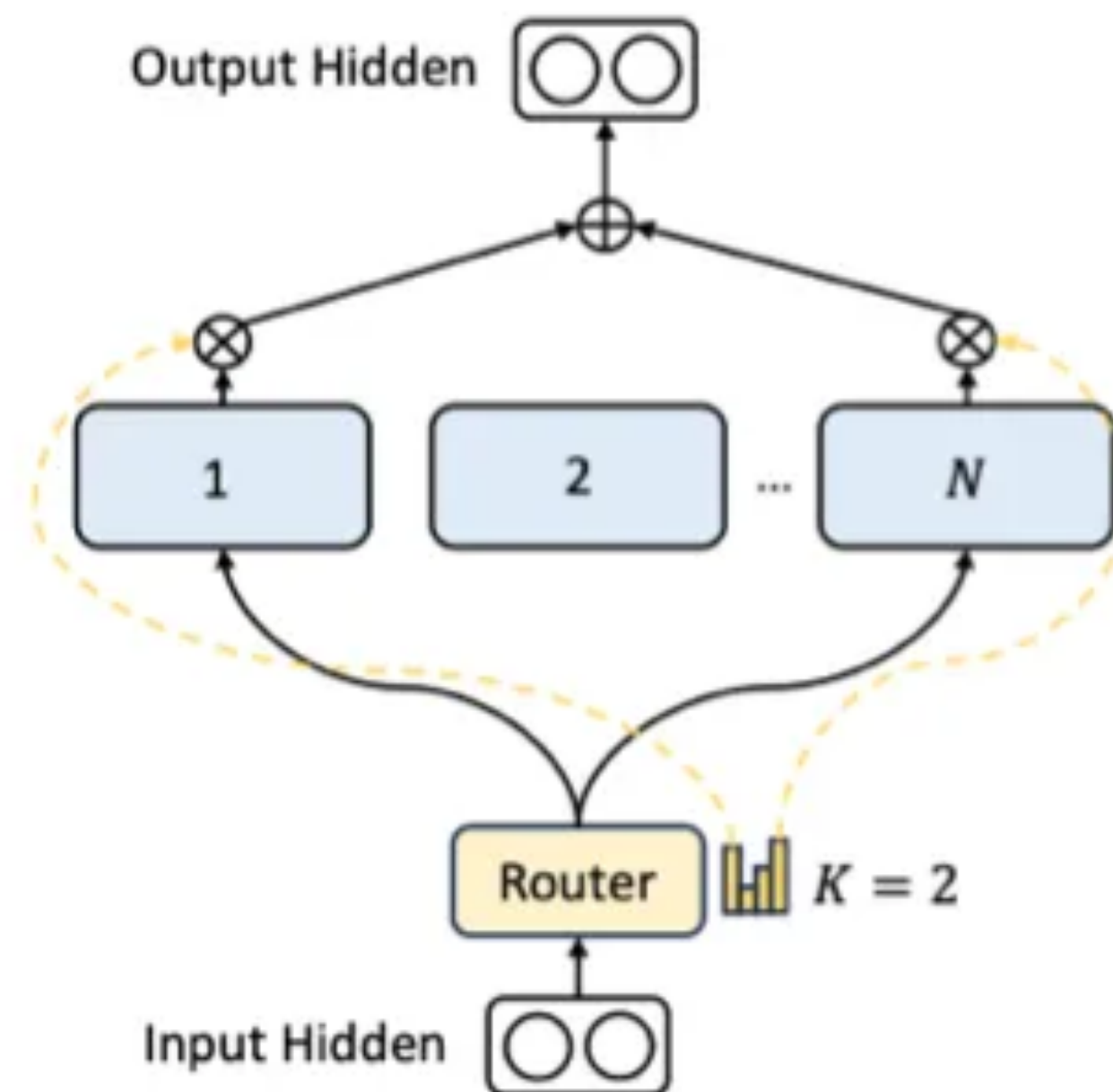
Mixture of Experts

- Allows for large capacity with low computational cost per token
- Router that selects which expert (individual neural networks) to use



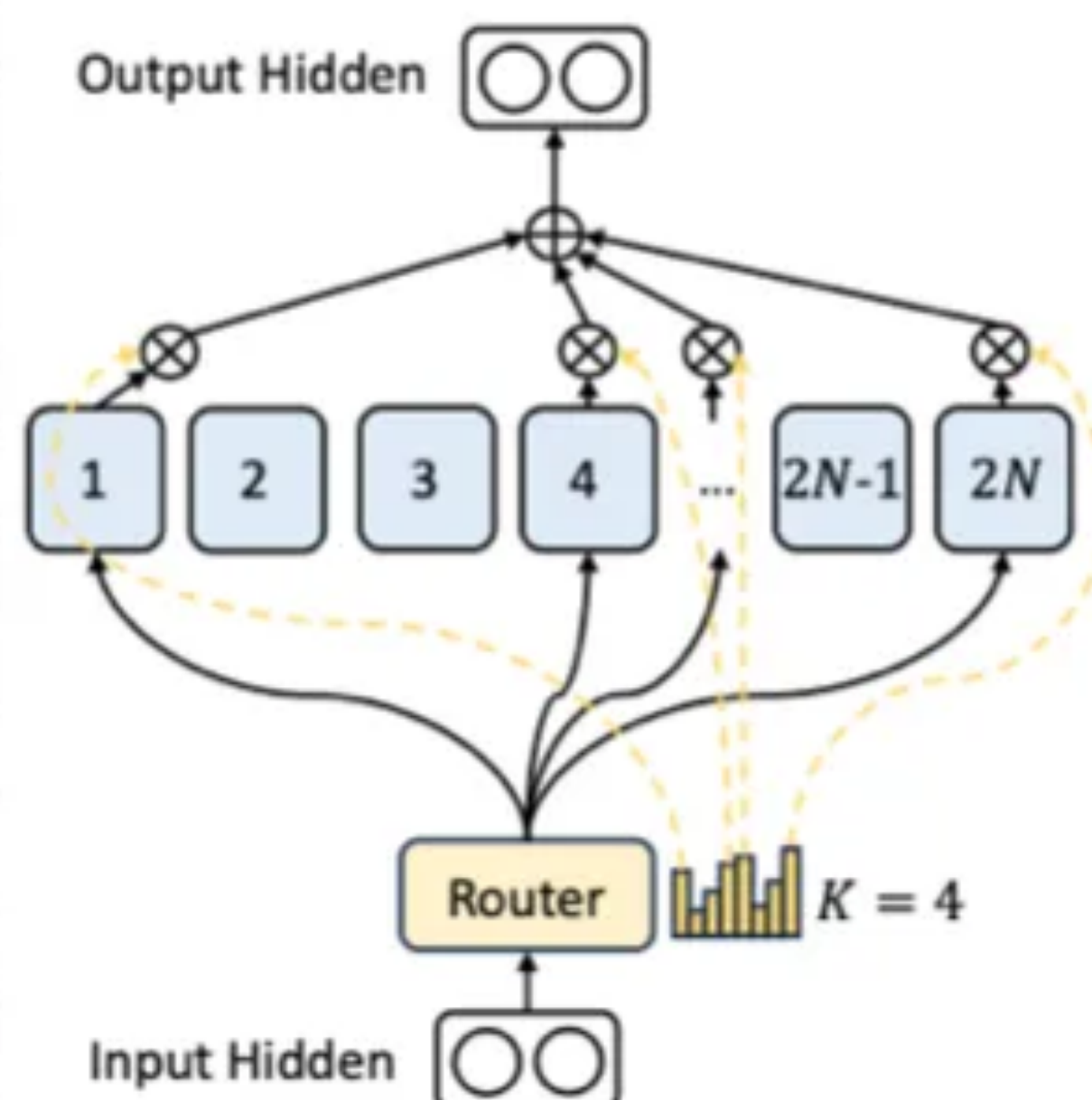
Mixture of experts over time

Early MoE: Has bigger and fewer experts, and activates only a few experts (here: 2)



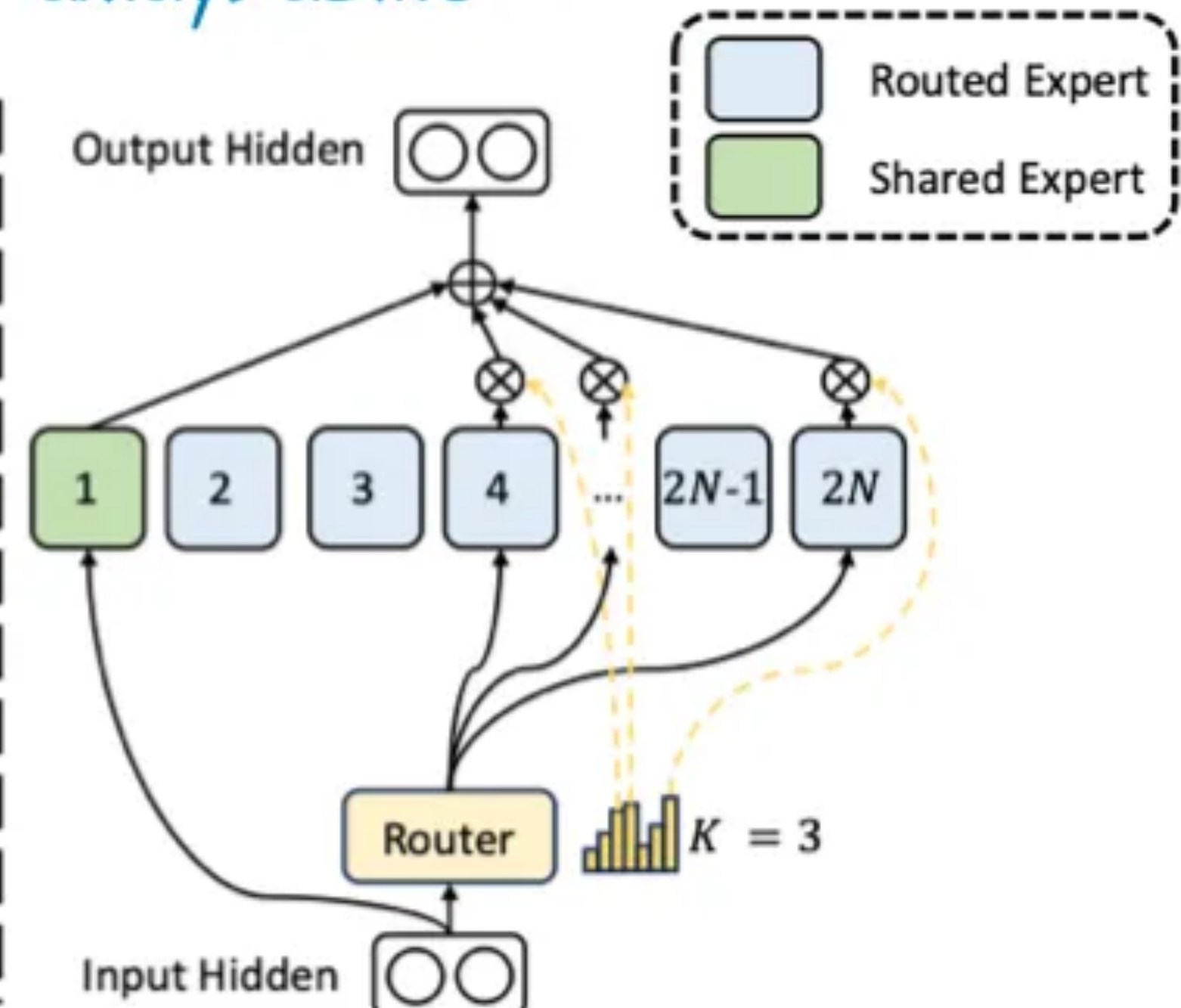
(a) Conventional Top-2 Routing

Fine-grained MoE uses more but smaller experts, and activates more experts (here: 4)



(b) + Fine-grained Expert Segmentation

MoE with shared expert: also uses many small experts, but adds a shared expert that is always active

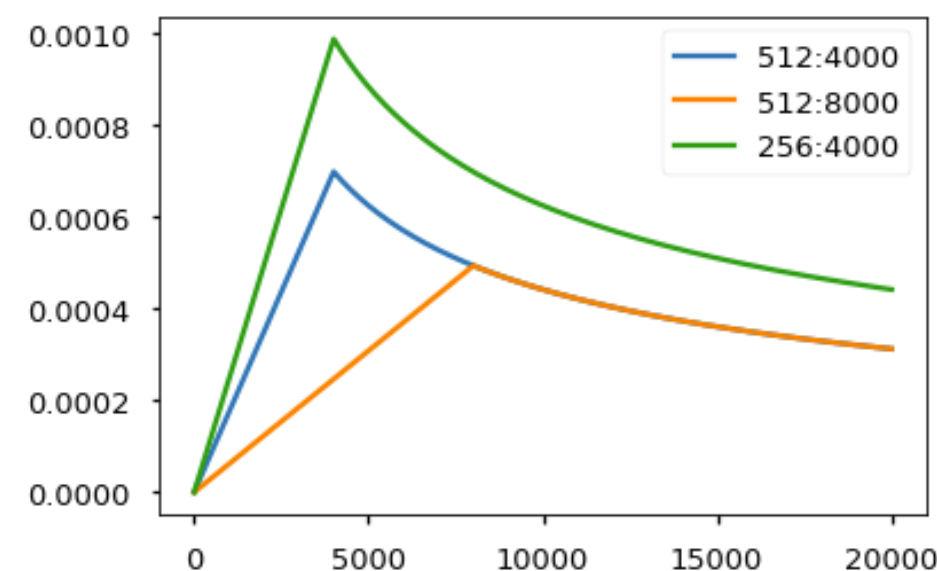


(c) + Shared Expert Isolation (DeepSeekMoE)

Optimizer

- SGD: Update in the direction of reducing loss
- Adam: Add momentum and normalize by the stddev of the outputs
- AdamW: properly applies weight decay for regularization to Adam
- Adam with learning rate schedule:

$$\text{learning_rate} = d_{\text{model}}^{-0.5} \cdot \min(\text{step_num}^{-0.5}, \text{step_num}^{-0.5} \cdot \text{warmup_steps}^{-1.5})$$



Muon Optimizer

- Orthogonalize the update

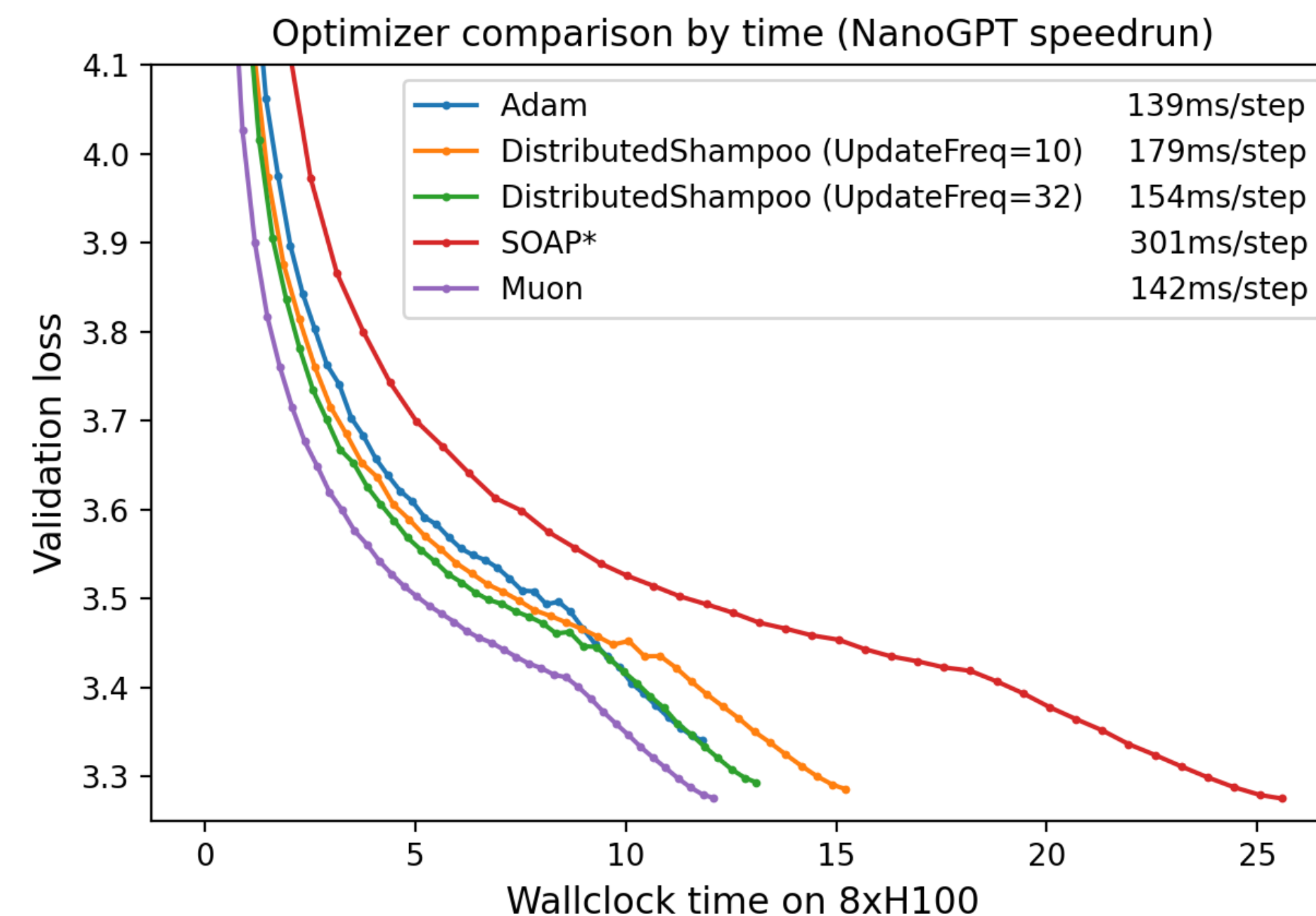
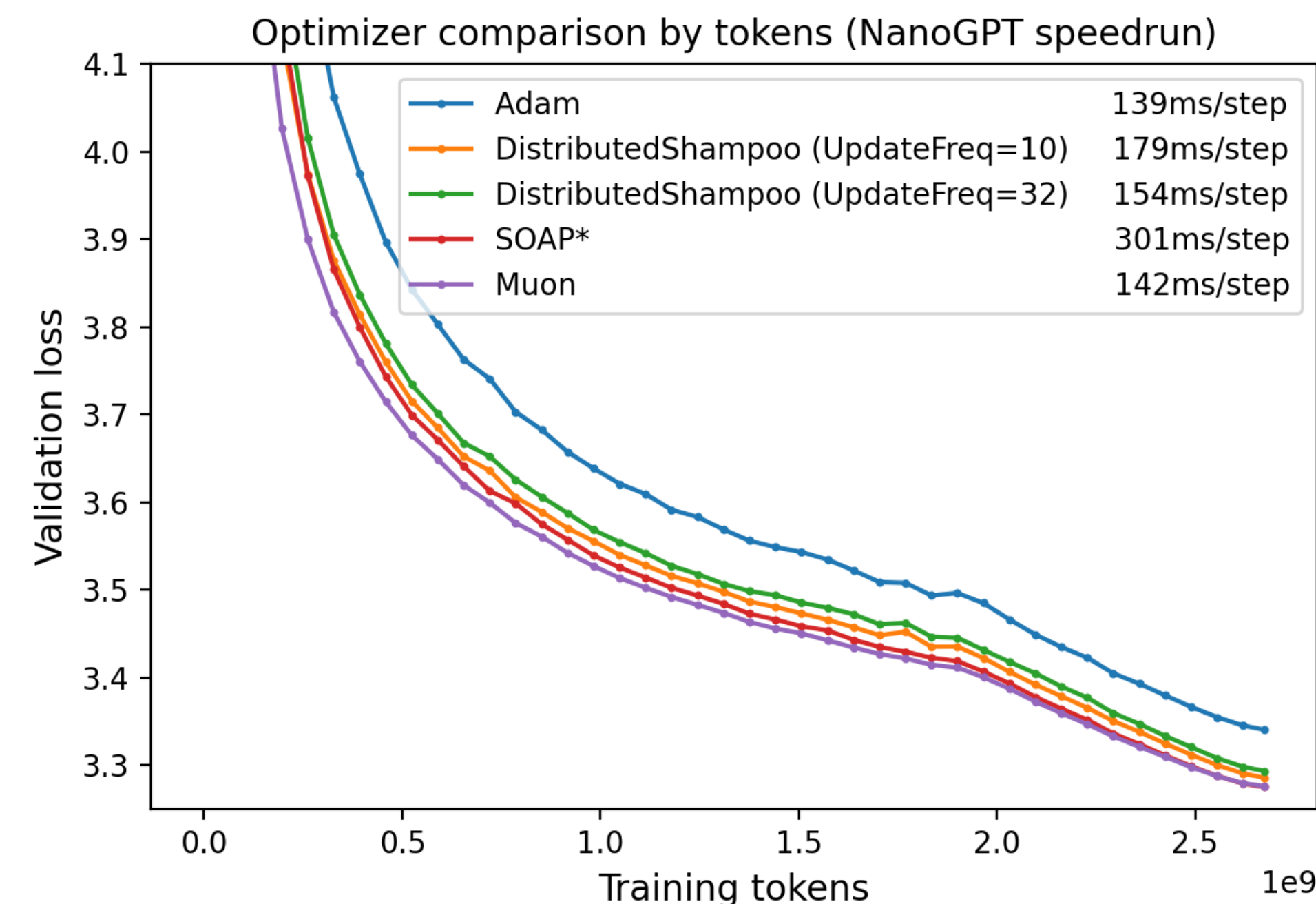
Algorithm 2 Muon

Require: Learning rate η , momentum μ

- 1: Initialize $B_0 \leftarrow 0$
 - 2: **for** $t = 1, \dots$ **do**
 - 3: Compute gradient $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$
 - 4: $B_t \leftarrow \mu B_{t-1} + G_t$
 - 5: $O_t \leftarrow \text{NewtonSchulz5}(B_t)$
 - 6: Update parameters $\theta_t \leftarrow \theta_{t-1} - \eta O_t$
 - 7: **end for**
 - 8: **return** θ_t
-

- Use for weight matrices, use AdamW for embeddings

<https://kellerjordan.github.io/posts/muon/>



*SOAP is under active development. Future versions will significantly improve the wallclock overhead.

Resources

- LLM architectures
 - <https://magazine.sebastianraschka.com/p/the-big-llm-architecture-comparison>
 - <https://magazine.sebastianraschka.com/p/a-dream-of-spring-for-open-weight>
- Andrej Karpathy - NanoGPT journey
 - <https://github.com/karpathy/nanochat/discussions/481>